

Ridge Regression (L2 Regularization)

Definition

Ridge Regression is a type of Linear Regression that reduces overfitting by adding an L2 regularization term (the square of the weights) to the cost function. It shrinks the model's coefficients but does not usually make them exactly zero.

Why do we need Ridge Regression?

When Linear Regression overfits the training data, it often learns very large weights. Ridge Regression penalizes these large weights, making the model simpler and reducing overfitting.

Cost Function

Linear Regression

$$Cost = MSE$$

Ridge Regression

$$Cost = MSE + \lambda \sum w_i^2$$

Lasso Regression (L1 Regularization)

Definition

Lasso Regression is a type of Linear Regression that reduces overfitting by adding an L1 regularization term (the absolute value of the weights) to the cost function. It can shrink some coefficients to exactly zero, automatically performing feature selection.

Why do we need Lasso Regression?

Sometimes datasets contain many irrelevant features. Lasso automatically identifies these unimportant features and removes them from the model.

Cost Function

$$Cost = MSE + \lambda \sum |w_i|$$

Elastic Net Regression

Definition

Elastic Net Regression is a regularized regression technique that combines both Ridge (L2) and Lasso (L1) regularization to reduce overfitting while also performing feature selection.

Simple Definition

Elastic Net combines the advantages of Ridge and Lasso Regression.

Cost Function

$$Cost = MSE + \lambda_1 \sum |w_i| + \lambda_2 \sum w_i^2$$

Evaluation Metrics

Confusion Matrix

Definition

A Confusion Matrix is a table used to evaluate the performance of a classification model by comparing the model's predictions with the actual values.

Simple Definition

A Confusion Matrix shows how many predictions were correct and how many were incorrect.

It is called **Confusion Matrix** because it shows where the model gets confused.

1. True Positive (TP)

Definition

The model predicts Positive, and the actual class is also Positive.

Simple Definition

The model correctly predicts a positive case.

Example

Patient has cancer.

Model says

Cancer

Correct.

TP increases.

2. True Negative (TN)

Definition

The model predicts Negative, and the actual class is also Negative.

Simple Definition

The model correctly predicts a negative case.

Example

Healthy patient.

Model says

Healthy

Correct.

TN increases.

3. False Positive (FP)

Definition

The model predicts Positive, but the actual class is Negative.

Simple Definition

The model raises a false alarm.

Example

Healthy patient.

Model says

Cancer

Wrong.

This is a False Positive.

Memory Trick

False Positive = "False Alarm"

4. False Negative (FN)

Definition

The model predicts Negative, but the actual class is Positive.

Simple Definition

The model misses a real positive case.

Example

Patient has cancer.

Model says

Healthy

Very dangerous.

Why are False Negatives dangerous?

Suppose

Airport Security

Actual

Terrorist

Prediction

Safe

This is

False Negative.

Very dangerous.

Cancer Detection

Actual

Cancer

Prediction

Healthy

Again

False Negative.

Life-threatening.

Why are False Positives bad?

Suppose

Healthy patient.

Model predicts

Cancer

The patient undergoes unnecessary tests, stress, and cost.

Bad, but generally less dangerous than missing a real cancer case.

Summary Table

Term	Definition	Easy Meaning
TP	Predicted Positive & Actually Positive	Correct Positive
TN	Predicted Negative & Actually Negative	Correct Negative
FP	Predicted Positive but Actually Negative	False Alarm
FN	Predicted Negative but Actually Positive	Missed Detection

Bank Fraud

Actual Predicted Result

Fraud Fraud TP

Fraud Genuine FN

Genuine Fraud FP

Genuine Genuine TN

1. Accuracy

Definition

Accuracy is the ratio of correctly predicted observations to the total number of observations.

Simple Definition

Accuracy tells us what percentage of predictions made by the model are correct.

Formula

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Problem with Accuracy

Suppose

1000 patients

990 Healthy

10 Cancer

Model predicts

Everyone is Healthy

Accuracy

$$\frac{990}{1000} = 99\%$$

Amazing?

No.

It missed **every cancer patient**.

Accuracy completely failed.

2. Precision

Definition

Precision is the ratio of correctly predicted positive observations to all observations predicted as positive.

Simple Definition

Precision answers: "Out of all the positive predictions made by the model, how many were actually positive?"

Formula

$$Precision = \frac{TP}{TP + FP}$$

Example

Model predicted

50 patients have cancer.

Actually

40 had cancer.

10 were healthy.

Therefore

TP =40

FP =10

Precision

$$\frac{40}{40 + 10} = 0.8$$

80%

Meaning

When the model predicts **Cancer**, it is correct **80%** of the time.

3. Recall (Sensitivity)

Definition

Recall is the ratio of correctly predicted positive observations to all actual positive observations.

Simple Definition

Recall answers: "Out of all the actual positive cases, how many did the model successfully find?"

Formula

$$Recall = \frac{TP}{TP + FN}$$

Memory Trick

Think

"How many real positives did the model recall (find)?"

Example

Suppose

50 people actually have cancer.

Model detected

45

Missed

5

Therefore

TP =45

FN =5

Recall

$$\frac{45}{45 + 5} = 90\%$$

Easy Memory Trick

Precision

Imagine you're shooting arrows.



XX

X

Few arrows,

but all hit the target.

Precision is high.

Recall

Imagine catching fish.

There are

100 fish.

You caught

98.

Excellent Recall.

Maybe

you also caught

some trash.

Precision may be lower.

4. Specificity

Definition

Specificity is the ratio of correctly predicted negative observations to all actual negative observations.

Simple Definition

Specificity tells us how well the model identifies negative cases correctly.

Formula

$$Specificity = \frac{TN}{TN + FP}$$

4. Specificity

Definition

Specificity is the ratio of correctly predicted negative observations to all actual negative observations.

Simple Definition

Specificity tells us how well the model identifies negative cases correctly.

Formula

$$Specificity = \frac{TN}{TN + FP}$$

Example

Suppose

Healthy people

100

Correctly predicted

95

Wrongly predicted cancer

5

Specificity

$$\frac{95}{95 + 5}$$

95%

High Specificity Means

Few False Positives.

Recall vs Specificity

Recall

Looks at Positive Class

Uses TP and FN

High Recall → Low FN

Specificity

Looks at Negative Class

Uses TN and FP

High Specificity → Low FP

5. F1 Score

Sometimes

Precision is high.

Recall is low.

Which model is better?

Need one metric.

Definition

F1 Score is the harmonic mean of Precision and Recall. It provides a balanced measure when both metrics are important.

Simple Definition

F1 Score balances Precision and Recall into a single value.

Formula

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Why Harmonic Mean?

Suppose

Precision

100%

Recall

0%

Average

would be

50%

which is misleading.

The harmonic mean heavily penalizes low values, so the F1 Score becomes **0**, correctly reflecting poor overall performance.

Example

Precision

80%

Recall

90%

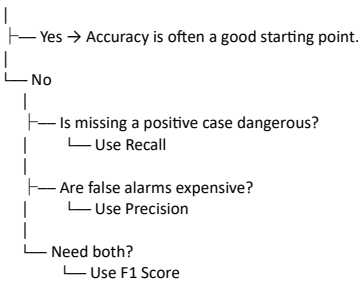
F1

$$2 \times \frac{0.8 \times 0.9}{0.8 + 0.9} = 0.847$$

84.7%

How to Decide Which Metric to Use

Is the dataset balanced?



Threshold is the cutoff value above which we predict Class 1 and below which we predict Class 0.

Example 1: Cancer Detection

Probability

0.35

Using threshold

0.5

Prediction

Healthy ❌

But there is still a **35% chance** of cancer.

Doctors may prefer

Threshold

0.2

to avoid missing patients.

ROC Curve

Definition

The ROC (Receiver Operating Characteristic) Curve is a graph that shows the performance of a classification model at different classification thresholds by plotting the True Positive Rate against the False Positive Rate.

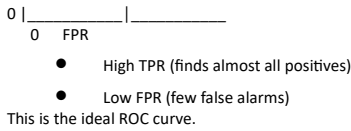
Simple Definition

ROC Curve shows how the model performs as we change the threshold.

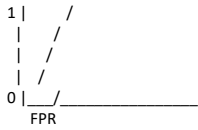
Perfect Model

TPR





Random Guessing
TPR



This diagonal line represents **random guessing**.

Better ROC Curve

The closer the curve is to the **top-left corner**, the better the classifier

AUC

Definition

AUC (Area Under the ROC Curve) measures the overall ability of a classifier to distinguish between positive and negative classes.

Simple Definition

AUC tells us how good the model is at separating the two classes.

Range
 $0 \leq AUC \leq 1$

Interpretation

AUC	Meaning
1.0	Perfect Classifier
0.9 - 0.99	Excellent
0.8 - 0.9	Very Good
0.7 - 0.8	Good
0.6 - 0.7	Average
0.5	Random Guessing
<0.5	Worse than Random

Logistic Regression

Logistic Regression is a machine learning algorithm used for classification problems. Instead of predicting numbers, it predicts the probability that an input belongs to a particular class (such as Yes/No, Spam/Not Spam, Fraud/Not Fraud).

Logistic Regression Equation

The model works in two steps.

Step 1

Compute

$$z = wx + b$$

Step 2

Pass it through the Sigmoid Function

$$P(y = 1 | x) = \frac{1}{1 + e^{-(wx+b)}}$$

This is the final Logistic Regression equation.

Why Do We Need a Cost Function?

Definition

A cost function is a mathematical function that measures how far the model's predictions are from the actual values.

Simple Definition

The cost function tells us how good or bad the model is. Lower cost means a better model.

3. Maximum Likelihood Estimation (MLE)

Definition

Maximum Likelihood Estimation (MLE) is a statistical method used to estimate model parameters by choosing the values that maximize the probability of observing the given training data.

Simple Definition

MLE finds the weights that make the observed data most likely.

Real-Life Intuition

Suppose you have two doctors.

Doctor A says

Cancer Probability = 0.98

Doctor B says

Cancer Probability = 0.55

Suppose the patient actually has cancer.

Which doctor seems more believable?

Doctor A.

Because Doctor A assigned a much higher probability to the correct outcome.

MLE simply says:

Choose the model that assigns the highest probability to the true labels.

8. Binary Cross Entropy (Log Loss)

Definition

Binary Cross Entropy (BCE), also called Log Loss, is the loss function used in Logistic Regression to measure the difference between predicted probabilities and actual binary labels.

Simple Definition

Binary Cross Entropy heavily penalizes confident wrong predictions and gives low loss to confident correct predictions.

Formula

$$J = -\frac{1}{n} \sum [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

1. Goal of Training

Definition

Training is the process of finding the optimal values of the model parameters (weights and bias) that minimize the loss function.

2. Forward Propagation

Forward propagation means computing the prediction.

It consists of three steps.

Step 1

Compute

$$z = wx + b$$

Step 2

Apply Sigmoid

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Step 3

Compute BCE Loss

$$J = -\frac{1}{n} \sum [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Now we know how wrong our prediction is.

Gradient

Definition

The gradient is the derivative of the loss function with respect to the model parameters. It indicates the direction and rate at which the loss changes.

1. Complete Logistic Regression Algorithm

Step-by-Step Algorithm

Step 1

Initialize

$$w = 0, b = 0$$

Step 2

For each training sample

Compute

$$z = wx + b$$

Step 3

Apply Sigmoid

$$\hat{y} = \frac{1}{1 + e^{-z}}$$

Step 4

Compute Binary Cross Entropy

$$J = -\frac{1}{n} \sum [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Step 5

Compute Gradients

$$dw = \frac{1}{n} \sum (\hat{y} - y)x$$
$$db = \frac{1}{n} \sum (\hat{y} - y)$$

Step 6

Update

$$w = w - adw$$
$$b = b - adb$$

Step 7

Repeat until convergence.

. Feature Scaling

Standardization

$$z = \frac{x - \mu}{\sigma}$$

Min-Max Scaling

$$x' = \frac{x - \min}{\max - \min}$$

1. What is a Decision Tree?

Definition

A Decision Tree is a supervised machine learning algorithm that predicts an output by recursively splitting the dataset into smaller subsets based on feature values, forming a tree-like structure of decisions.

Simple Definition

A Decision Tree makes predictions by asking a sequence of Yes/No questions until it reaches a final decision.

Parametric Model

Definition

A parametric model has a fixed number of parameters regardless of the dataset size.

Example

Linear Regression

$$y = wx + b$$

Only

- weights
- bias

Fixed.

Non-Parametric Model

Definition

A non-parametric model does not assume a fixed mathematical form, and the number of parameters can grow with the size and complexity of the data.

Decision Tree is

✔ Non-Parametric.

3. Pure vs Impure Nodes

Pure Node

A node is **pure** if all samples belong to the same class.

Goal of a Decision Tree

The goal is to convert

Mixed Data

↓

Less Mixed

↓

Almost Pure

↓

Completely Pure

Entropy: **Entropy** measures the amount of uncertainty or randomness in a dataset.

Simple Definition

Entropy tells us how disordered or uncertain the data is.

6. Formula of Entropy

Definition Formula

For binary classification,

$$Entropy(S) = -P_1 \log_2(P_1) - P_2 \log_2(P_2)$$

More generally,

$$Entropy(S) = - \sum_{i=1}^n P_i \log_2(P_i)$$

where:

- P_i = Probability of class i
- n = Number of classes

8. Information Gain

Now comes the most important concept.

Definition

Information Gain measures the reduction in entropy after splitting the dataset using a particular feature.

Simple Definition

Information Gain tells us how much uncertainty is reduced after making a split.

Formula

$$IG = Entropy(Parent) - Weighted Entropy(Children)$$

Expanded form

$$IG = Entropy(S) - \sum_{i=1}^k \frac{|S_i|}{|S|} Entropy(S_i)$$

where:

- S = Parent dataset
- S_i = Child dataset
- $|S_i|$ = Number of samples in child node
- $|S|$ = Total number of samples

Gini Index :

Definition

Gini Index measures the probability of incorrectly classifying a randomly chosen sample if labels are assigned according to the class distribution in the node.

Simple Definition

Gini tells us how mixed the classes are. Lower Gini means purer data.

3. Formula

For binary classification

$$Gini = 1 - P_1^2 - P_2^2$$

General Formula

$$Gini = 1 - \sum_{i=1}^n P_i^2$$

where

- P_i = Probability of class i

6. Stopping Criteria

To prevent overfitting, we stop tree growth.

These are called

Stopping Criteria

(a) Maximum Depth

Definition

Maximum number of levels allowed.

Example

Depth = 3

Interview Point

Large depth

↓

Overfitting

Small depth

↓

Underfitting

(b) Minimum Samples Split

Definition

Minimum number of samples required before a node can be split.

(c) Minimum Samples Leaf

Definition

Minimum number of samples every leaf must contain.

Hyperparameters

max_depth

Controls tree depth.

DecisionTreeClassifier(max_depth=5)

min_samples_split

Minimum samples before splitting.

min_samples_leaf

Minimum samples in every leaf.

criterion

Split criterion.

criterion="gini"

8. Pruning

Stopping criteria stop the tree **before** it grows too much.

But

What if

the tree has already grown?

We remove unnecessary branches.

This is called

Pruning

Pruning is the process of removing unnecessary branches from a Decision Tree to reduce overfitting and improve generalization.

Pre-Pruning

Stop growth early.

Example

Reached max depth

↓

Stop

Advantages

✓ Fast

✓ Small tree

Disadvantages

May stop too early.

Underfitting.

Post-Pruning

Grow complete tree.

Then

Remove weak branches.

Example

Huge Tree

↓

Evaluate Branch

↓

No improvement

↓

Delete Branch

Advantages

Usually gives better accuracy.

Disadvantages

More computation.

Final Hierarchy :
Input Dataset
↓
Compute Gini/Entrop
↓
Try Every Feature
↓
Find Best Split
↓
Split Dataset
↓
Repeat Recursively
↓
Stopping Condition
↓
Return Tree

10. Assumptions

Unlike Linear Regression

Decision Trees make very few assumptions.

No assumption of

- Linearity
- Normality
- Equal Variance

They naturally handle nonlinear relationships.

Random Forest

Decision Tree
↓
Problem: Overfitting
↓
Can we use Multiple Trees?
↓
Random Forest
↓
Bagging
↓
Bootstrap Sampling
↓
Random Feature Selection
↓
Majority Voting
↓
Final Prediction

What is Ensemble Learning?

An **ensemble** means a **group**.

Instead of relying on **one machine learning model**, we combine **multiple models** to make one final prediction.

Think of it like this:

Many weak opinions together can form one strong decision

Why use Ensemble Learning?

A single model may:

- Overfit
- Underfit
- Miss patterns
- Be unstable

Combining multiple models usually:

- Improves accuracy
- Reduces errors
- Makes predictions more stable
- Improves generalization

BAGGING:

Bagging trains **many models independently and in parallel**, then combines their predictions.

Each model learns from a **different random subset** of the training data.

Why is it called Bootstrap?

Bootstrap means:

Randomly sample the dataset **with replacement**.

Boosting

Boosting works very differently.

Instead of training models independently,

each new model **learns from the mistakes made by the previous model**.

Definition

Boosting builds models **sequentially**.

Every new model focuses more on the samples that previous models predicted incorrectly.

Popular Boosting Algorithms

AdaBoost

Adaptive Boosting.

Increases the weight of incorrectly classified samples so the next model pays more attention to them.

Gradient Boosting

Instead of adjusting sample weights, it trains the next model to predict the **errors (residuals)** made by the previous model.

XGBoost

The most popular boosting algorithm in competitions and industry.

Adds:

- Regularization
- Parallelized tree construction
- Missing value handling
- Tree pruning
- Better optimization

Very fast and highly accurate.

LightGBM

Developed by Microsoft.

Optimized for:

- Large datasets
- Faster training
- Lower memory usage

CatBoost

Developed by Yandex.

Excellent for datasets with many **categorical features**.

Requires minimal preprocessing.

Random Forest

Random Forest is simply

Bagging + Decision Trees

It builds hundreds of decision trees

What is Random Forest?

Definition

Random Forest is an ensemble learning algorithm that builds multiple Decision Trees using randomly selected training samples and randomly selected features, then combines their predictions to produce the final output.

Simple Definition

Random Forest is a collection of many Decision Trees that work together to make better predictions.

3. Why is "Random" Used?

This is an interview favorite.

Random Forest introduces randomness in **two places**:

1. Random Data

Every tree gets a different dataset.

2. Random Features

Every split considers only a random subset of features.

At every split, Random Forest randomly selects only a subset of the available features and finds the best split **within that subset**.

4. Complete Random Forest Training Algorithm

Step 1

Input training dataset.

↓

Step 2

Create a bootstrap sample.

↓

Step 3

Grow a Decision Tree.

At every node:

- Randomly choose m features out of d .
- Find the best split using only those m features.

↓

Step 4

Repeat until

T trees are built.

↓

Step 5

Combine predictions.

Classification

↓

Majority Vote

Regression

↓

Average

5. Hyperparameters

(a) n_estimators

Number of trees.

RandomForestClassifier(

 n_estimators=100

)

More trees

↓
Better stability
↓
More computation
(f) **bootstrap**
bootstrap=True
Uses bootstrap sampling.
If False,
every tree gets the full dataset.
(c) **max_depth**
Maximum tree depth.
Controls overfitting.

(d) **min_samples_split**
Minimum samples required before splitting.

(e) **min_samples_leaf**
Minimum samples allowed in a leaf.

Random Forest introduces randomness in **two ways**:

1. Bootstrap Sampling
2. Random Feature Selection

But one question remains...

How do we evaluate the model without creating a validation set?

The answer is:

Out-of-Bag (OOB) Error

2. What is Out-of-Bag (OOB) Error?

Recall:

Each tree trains on only about **63.2%** of the unique samples.

The remaining **36.8%** are never seen by that tree.

These unseen samples are called

Out-of-Bag Samples

Formula

$$OOB\ Error = \frac{\text{Number of Incorrect OOB Predictions}}{\text{Total Number of OOB Predictions}}$$

OOB Accuracy

$$OOB\ Accuracy = 1 - OOB\ Error$$

9. When Should We Use Random Forest?

Use when

- ✓ Tabular data
- ✓ Medium-sized datasets
- ✓ Nonlinear relationships
- ✓ Mixed numerical and categorical features (after appropriate encoding if required)
- ✓ High accuracy needed

Avoid when

- ✗ Very large datasets with strict latency constraints
- ✗ Deep learning problems
- ✗ Images
- ✗ Audio
- ✗ NLP

If you're already using a Random Forest, **OOB score** is a quick way to estimate performance.

For final model selection or comparing different algorithms, **cross-validation** is generally preferred.

Things to Remember Forever

- Around **36.8%** of samples are **Out-of-Bag** for each tree.
- **OOB Error** provides an internal estimate of generalization performance without creating a separate validation set.
- Random Forest computes **feature importance** by averaging impurity reductions across all trees.

Boosting:

Boosting is an ensemble learning technique where weak learners are trained sequentially, with each learner focusing more on the errors made by previous learners.

Simple Definition

Every new model tries to correct the mistakes of the previous model.

A **Weak Learner** is a model that performs only slightly better than random guessing.

A Decision Tree with

max_depth=1

is called a

Decision Stump

It is the most common weak learner used in AdaBoost.

A Decision Stump has only

- One split
- Two leaves

Training Data

↓
Weak Learner 1

↓
Find Mistakes

↓
Weak Learner 2

↓
Find Remaining Mistakes
↓
Weak Learner 2
↓
Combine All Learners
↓
Strong Model

10. Characteristics of Boosting

- ✓ Sequential learning
- ✓ Weak learners
- ✓ Error correction
- ✓ High accuracy
- ✓ Lower bias
- ✓ Sensitive to noise

11. Advantages

- ✓ Higher accuracy than Bagging in many structured-data tasks
- ✓ Reduces bias
- ✓ Works well on complex datasets
- ✓ Excellent performance on tabular data

12. Disadvantages

- ✗ Slower training
- ✗ Cannot easily parallelize training
- ✗ More sensitive to noisy labels and outliers
- ✗ Requires careful hyperparameter tuning

ADABOOST:

Random Forest

↓

Every sample is treated equally.

AdaBoost

↓

Every sample has a **weight**.

Initially,

all samples have equal importance.

After every iteration,

incorrectly classified samples receive **higher weight**.

Correctly classified samples receive **lower weight**.

Thus,

the next learner focuses on the difficult examples.

. What are Sample Weights?

Suppose we have 5 training samples.

Sample Class

A	Yes
B	Yes
C	No
D	Yes
E	No

Initially,

every sample is equally important.

Since

$$N = 5$$

Initial weight

$$w_i = \frac{1}{N}$$

Therefore

$$w_i = \frac{1}{5} = 0.2$$

Weights

Sample Weight

A	0.2
B	0.2
C	0.2
D	0.2
E	0.2

Notice

Total weight

$$0.2 + 0.2 + 0.2 + 0.2 + 0.2 = 1$$

Weights always sum to 1.

3. Train First Weak Learner

Suppose we train

a **Decision Stump**.

Predictions

Sample Actual Prediction

A	Yes	Yes ✓
B	Yes	Yes ✓

Sample Actual Prediction

C	No	Yes ❌
D	Yes	Yes ✅
E	No	No ✅

Only

Sample C
is incorrect.

4. Weighted Error

Unlike normal accuracy,
AdaBoost uses
Weighted Error.

Definition

Weighted Error is the sum of the weights of incorrectly classified samples.

Formula

$$\epsilon = \sum_{i=1}^N w_i \cdot I(y_i \neq \hat{y}_i)$$

where

- w_i = sample weight
- $I(\cdot)$ = indicator function
$$I = \begin{cases} 1, & \text{wrong prediction} \\ 0, & \text{correct prediction} \end{cases}$$

Example

Only Sample C is wrong.

Weight

0.2

Therefore

$$\epsilon = 0.2$$

Notice

AdaBoost doesn't care how many samples are wrong.

It cares about

how much total weight is wrong.

5. Learner Weight (Alpha)

Now we measure

how good

this weak learner is.

This is called

Alpha

Formula

$$\alpha = \frac{1}{2} \ln \left(\frac{1-\epsilon}{\epsilon} \right)$$

where

- ϵ = weighted error

Example

Suppose

$$\epsilon = 0.2$$

Then

$$\alpha = \frac{1}{2} \ln \left(\frac{0.8}{0.2} \right) = \frac{1}{2} \ln 4$$

Since

$$\ln 4 = 1.386$$

Therefore

$$\alpha = 0.693$$

Interpretation of Alpha

Large Alpha

↓

Excellent learner.

Small Alpha

↓

Weak learner.

Suppose

Weighted Error

$$0.05$$

Alpha

Huge.

Weighted Error

$$0.45$$

Alpha

Very small.

Weighted Error

$$0.5$$

Then

$$\alpha = 0$$

Meaning

The learner is no better than random guessing.

Such a learner contributes nothing.

If

$$\epsilon > 0.5$$

Alpha becomes negative.

This means the learner performs worse than random guessing.

In practice, AdaBoost assumes the weak learner performs better than random. If a learner has error greater than 0.5 in binary classification, it is usually discarded or retrained.

6. Why This Formula?

Let's analyze it.

Case 1

Perfect learner

$$\epsilon = 0$$

Denominator becomes zero.

Mathematically,

$$\alpha \rightarrow \infty$$

Perfect learner receives extremely high importance.

(In practice, implementations stop early or clip the value to avoid division by zero.)

Case 2

Random learner

$$\epsilon = 0.5$$

$$\alpha = 0$$

No contribution.

Case 3

Bad learner

$$\epsilon = 0.7$$

$$\alpha < 0$$

Bad learner receives negative weight.

Graph

Alpha

↑

|

| /

| /

| /

| /

| _/

----->

Weighted Error

0 0.5 1

Alpha decreases

as error increases.

7. Update Sample Weights

This is the heart of AdaBoost.

Correctly classified samples

↓

Decrease weight.

Incorrectly classified samples

↓

Increase weight.

General update rule:

$$w_i \leftarrow w_i \exp(-\alpha y_i h(x_i))$$

where

- $y_i \in \{-1, +1\}$ is the true label.
- $h(x_i) \in \{-1, +1\}$ is the prediction of the weak learner.

This compact formula works because:

- If prediction is correct, $y_i h(x_i) = +1$, so the factor is $e^{-\alpha}$ (weight decreases).
- If prediction is wrong, $y_i h(x_i) = -1$, so the factor is $e^{+\alpha}$ (weight increases).

For intuition, you can think of it as:

Correct prediction

↓

Multiply by

$$e^{-\alpha}$$

Wrong prediction

↓

Multiply by

$$e^{+\alpha}$$

8. Numerical Example

Current weights

Sample Weight

A 0.2
B 0.2
C 0.2
D 0.2
E 0.2

Alpha 0.693

Since $e^{0.693} = 2$

and $e^{-0.693} = 0.5$

Updated weights (before normalization)

Sample Correct? New Weight

A Yes $0.2 \times 0.5 = 0.1$
B Yes 0.1
C No $0.2 \times 2 = 0.4$
D Yes 0.1
E Yes 0.1

Total 0.8

But weights must sum to 1.

Normalize by dividing each weight by 0.8:

Sample Normalized Weight

A 0.125
B 0.125
C 0.50
D 0.125
E 0.125

Notice:

Sample C now has 50% of the total weight.

The next Decision Stump will pay much more attention to it.

9. Complete AdaBoost Algorithm

Initialize Equal Weights

Train Decision Stump

Compute Weighted Error (ϵ)

Compute Alpha (α)

Update Sample Weights

Normalize Weights

Repeat

Formula Sheet

Initial Weight

$$w_i = \frac{1}{N}$$

Weighted Error

$$\epsilon = \sum w_i I(y_i \neq \hat{y}_i)$$

Learner Weight

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - \epsilon}{\epsilon} \right)$$

Weight Update

$$w_i \leftarrow w_i e^{-\alpha y_i h(x_i)}$$

Normalize

$$w_i = \frac{w_i}{\sum_j w_j}$$

Complexity

Suppose

- T = Number of weak learners
- n = Samples

Training

$$O(T \times C)$$

where C is the cost of training one weak learner.

Prediction

$$O(T)$$

for decision stumps.

1. What Happens After Training?

Suppose AdaBoost trains

- Stump 1
- Stump 2
- Stump 3

Each stump has learned different mistakes.

Question:

How do we combine them?

Unlike Random Forest,

where every tree gets one vote,

AdaBoost gives different importance to different learners.

2. Weighted Voting

Random Forest

Tree1 $\rightarrow$ 1 Vote

Tree2 $\rightarrow$ 1 Vote

Tree3 $\rightarrow$ 1 Vote

All trees are equal.

AdaBoost

Tree1 $\rightarrow$ Weight = 0.9

Tree2 $\rightarrow$ Weight = 0.2

Tree3 $\rightarrow$ Weight = 1.4

Tree 3 is much more reliable.

Therefore,

its vote should matter more.

AdaBoost Hyperparameters:

n_estimators

Number of weak learners.

AdaBoostClassifier(

n_estimators=100

)

More learners

$\downarrow$

Lower bias

$\downarrow$

Longer training.

learning_rate

Controls

how much each learner contributes.

learning_rate=0.5

Large learning rate

$\downarrow$

Faster learning

$\downarrow$

May overfit.

Small learning rate

$\downarrow$

Slower learning

$\downarrow$

Often better generalization.

estimator

Base learner.

Usually

Decision Stump.

DecisionTreeClassifier(max_depth=1)

11. Advantages

- ✓ Easy to understand
- ✓ High accuracy
- ✓ Reduces bias
- ✓ Works well on structured/tabular datasets
- ✓ Can convert weak learners into a strong learner

12. Disadvantages

- ✗ Sensitive to noisy data
- ✗ Sensitive to outliers
- ✗ Sequential training
- ✗ Slower than Bagging

Gradient Boosting:

1. Why Do We Need Gradient Boosting?

We already know
Random Forest
✓ Reduces Variance
AdaBoost
✓ Reduces Bias
Question
Can we do even better?
Yes.
Instead of changing sample weights,
we directly improve the model's predictions.
This is Gradient Boosting.

3. Main Idea

Suppose
Actual price
₹100
Our first model predicts
₹70
Error
30
Instead of predicting the original target again,
the next model predicts
30
the error.

New prediction
70+30=100
Perfect.
This is the main idea.
Every new tree predicts
the previous tree's mistakes.

4. What is a Residual?

Interview favorite.
Definition
A **Residual** is the difference between the actual value and the predicted value.
Formula

$$Residual = y - \hat{y}$$

where

- y =Actual value
- $\hat{y}$ =Predicted value

Example
Actual
100
Prediction
75
Residual

 $100 - 75 = 25$

Residual
25

Another sample
Actual
80
Prediction
90
Residual

 $80 - 90 = -10$

Negative residual
means
Prediction was too high.

5. Why Learn Residuals?

Suppose
Model 1
predicts

Actual	Prediction
100	80
50	45
70	60

Residuals

Actual	Prediction	Residual
100	80	20
50	45	5
70	60	10

Instead of learning
100

50

70

Tree 2 learns
20

5

10

which is usually much easier.

Why Gradient Descent?

$$\theta = \theta - \eta \nabla L$$

Gradient Boosting does something very similar.
Instead of updating parameters,
it adds
a new tree
that points
toward the direction of **maximum loss reduction**.
This direction is given by the **negative gradient** of the loss.
Hence
Gradient Boosting.

Term	Meaning
Residual	Actual – Prediction
Pseudo Residual	Negative gradient of the loss function
Stage-wise Learning	Adding one learner at a time
Base Learner	Usually a shallow Decision Tree
Gradient	Direction of steepest increase of a function
Negative Gradient	Direction of fastest decrease in loss

1. Complete Numerical Example

Suppose we want to predict house prices.
Dataset

House	Actual Price
A	100
B	120
C	150

Step 1 — Initial Prediction

For regression,
the initial prediction is the **mean**.

$$F_0 = \frac{100 + 120 + 150}{3} = 123.33$$

Every sample is initially predicted as

House	Prediction
A	123.33
B	123.33
C	123.33

Step 2 — Calculate Residuals

Residual

 $r = y - \hat{y}$

House	Actual	Prediction	Residual
A	100	123.33	-23.33
B	120	123.33	-3.33
C	150	123.33	26.67

These residuals become the target values for the first tree.

Step 3 — Train Tree 1

Suppose Tree 1 predicts

House	Predicted Residual
A	-20
B	-5
C	25

Step 4 — Learning Rate

Suppose

 $\eta = 0.1$

Updated prediction

 $F_1 = F_0 + 0.1h_1$

For House A

 $123.33 + 0.1(-20) = 121.33$

House B

 $123.33 + 0.1(-5) = 122.83$

House C

 $123.33 + 0.1(25) = 125.83$

Updated predictions

House	Prediction
A	121.33
B	122.83
C	125.83

Notice
The predictions moved **toward** the correct values.

Step 5 — New Residuals

Compute residuals again.

House A	$100 - 121.33 = -21.33$
House B	$120 - 122.83 = -2.83$
House C	$150 - 125.83 = 24.17$

Residuals are already smaller than before.

Step 6 — Train Tree 2

Tree 2 now learns
-21.33

-2.83

24.17
Suppose Tree 2 predicts
-20

-3

23
Update

$$F_2 = F_1 + 0.1h_2$$

Predictions improve again.

Step 7 — Repeat

Iteration
Prediction
|
Residual
↓
Tree
↓
Update
↓
Smaller Residual
↓
Repeat

Important Hyperparameters

n_estimators
Number of trees.
GradientBoostingRegressor(

n_estimators=300
)
More trees
↓
Lower bias
↓
Longer training
↓
Higher overfitting risk

learning_rate
Most important.
Typical values
0.01

0.05

0.1

0.2
Small learning rate
↓
Need more trees
↓
Usually better accuracy

max_depth
Depth of every tree.

8. Advantages

- ✓ Very high accuracy
- ✓ Supports regression
- ✓ Supports classification
- ✓ Supports custom differentiable loss functions
- ✓ Captures nonlinear relationships
- ✓ Works well on structured/tabular datasets

9. Disadvantages

- ✗ Slow training
- ✗ Sequential learning
- ✗ Sensitive to hyperparameters
- ✗ Can overfit
- ✗ Harder to interpret than a single tree

10. Random Forest vs Gradient Boosting

Eventually
Residuals become
-0.3

0.5

-0.2
Almost zero.
Training stops.

2. Complete Training Pipeline

Compute Mean
↓
Predict Mean
↓
Compute Residuals
↓
Fit Tree
↓
Multiply by Learning Rate
↓
Update Predictions
↓
Compute New Residuals
↓
Fit Next Tree
↓
Repeat

Small
↓
Underfitting
Large
↓
Overfitting

min_samples_split
Minimum samples required
before splitting.

min_samples_leaf
Minimum samples
inside leaf.

subsample
Very important.
Instead of using
100%
of data,
use
80%.
Example
subsample=0.8
Benefits
↓
Lower variance
↓
Less overfitting
This idea later evolves into **Stochastic Gradient Boosting**.

random_state
For reproducibility.

Random Forest	Gradient Boosting
Parallel training	Sequential training
Bootstrap sampling	Residual learning
Equal voting	Additive model
Mainly reduces variance	Reduces bias and can also reduce variance
Many deep trees	Usually many shallow trees
Faster training	Slower training
More robust to hyperparameter choices	Requires careful tuning

11. AdaBoost vs Gradient Boosting

AdaBoost	Gradient Boosting
Changes sample weights	Fits residuals/negative gradients
Exponential loss	Any differentiable loss
Mostly decision stumps	Shallow trees
Weighted voting	Additive prediction

AdaBoost	Gradient Boosting
Primarily classification	Regression & Classification

XGBOOST

XGBoost is still one of the strongest algorithms for **tabular data**, and understanding it makes LightGBM and CatBoost much easier.

Gradient Boosting is already very powerful.
Then why was XGBoost invented?

Because standard Gradient Boosting has several practical limitations.

Limitation 1 — Slow Training

Gradient Boosting builds trees
Every tree depends on the previous tree.
Training becomes slow.

Limitation 2 — Overfitting

Gradient Boosting has **no built-in regularization** like Linear Regression's Ridge or Lasso.

Limitation 3 — Memory

Large datasets
10 Million Rows
Training becomes expensive.

Limitation 5 — Parallelism

People often think
"Gradient Boosting cannot be parallelized."
That statement is only partially true.
The **trees themselves must still be built sequentially**, because each new tree depends on the previous one.
However,
inside a single tree,
operations like evaluating candidate feature splits can be parallelized.
XGBoost exploits this optimization, making training much faster.

2. What is XGBoost?

Definition

XGBoost (Extreme Gradient Boosting) is an optimized implementation of Gradient Boosting that adds regularization, efficient tree construction, parallel computation within tree building, sparse-data support, and several engineering optimizations to improve speed and accuracy.

XGBoost = Faster + Smarter + Regularized Gradient Boosting

If gradient bosting is normal car – 150km/hr
This is F1 car -> 350km/hr (faster, optimised)

Improvements over Gradient Boosting

Regularization

Controls
Tree complexity.
↓
Less overfitting.

Faster Split Finding

Instead of checking every split naively,
XGBoost uses optimized algorithms to evaluate candidate splits efficiently.

Parallel Processing

Tree building is still sequential,
but
finding the best split
for different features
can happen simultaneously.

Missing Values

Instead of removing
missing values,
XGBoost learns
which branch
missing values should follow.

Tree Pruning

Gradient Boosting
often grows a full tree
then trims it.
XGBoost grows the tree while considering whether additional splits are worthwhile, using its objective function to avoid unnecessary growth.

Cache Optimization

Modern CPUs
have cache memory.
XGBoost
uses cache-aware data structures
to reduce memory access.
Result
↓
Faster training.

6. Why is XGBoost So Accurate?

Suppose
Gradient Boosting

tries
1000 trees.
Some trees
may not help.
Some
increase overfitting.
XGBoost
penalizes
unnecessary complexity.
Therefore
Only useful trees
remain.

7. Main Features

- ✓ Regularization
- ✓ Missing Value Handling
- ✓ Parallel split finding
- ✓ Tree pruning
- ✓ Sparse data support
- ✓ High accuracy
- ✓ Fast training
- ✓ Scalable to millions of samples

8. Where is XGBoost Used?

Finance
Fraud Detection

Credit Risk

Loan Approval
Healthcare
Disease Prediction

Medical Diagnosis
Marketing
Customer Churn

Sales Forecasting
Cybersecurity
Intrusion Detection

Spam Detection
Competitions
Kaggle
For many years,
XGBoost dominated Kaggle competitions before deep learning and newer gradient boosting libraries became more common in certain domains.

9. Gradient Boosting vs XGBoost

Gradient Boosting	XGBoost
Basic algorithm	Optimized implementation
No explicit tree regularization	L1 & L2 regularization on tree complexity/leaf weights
Slower	Faster
Limited handling of missing values	Native missing-value handling
Fewer engineering optimizations	Cache-aware & scalable
More prone to overfitting	Better regularization

10. Complexity

Training
Approximately

$$O(T \cdot n \cdot d \cdot \log n)$$

where

- T =Trees
- n =Samples
- d =Features

Although the theoretical complexity is similar to Gradient Boosting,
the **constant factors are much smaller** due to engineering optimizations.

Prediction

$$O(T \log n)$$

11. Why Does XGBoost Need Mathematics?

Everything so far
was engineering.
The real innovation is
its objective function.
Unlike Gradient Boosting,
XGBoost doesn't simply fit residuals.
It minimizes

$$\boxed{\text{Objective} = \text{Loss} + \text{Regularization}}$$

This is the biggest mathematical difference.
We'll derive this completely.

Formula Sheet Gradient Boosting

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

XGBoost Objective

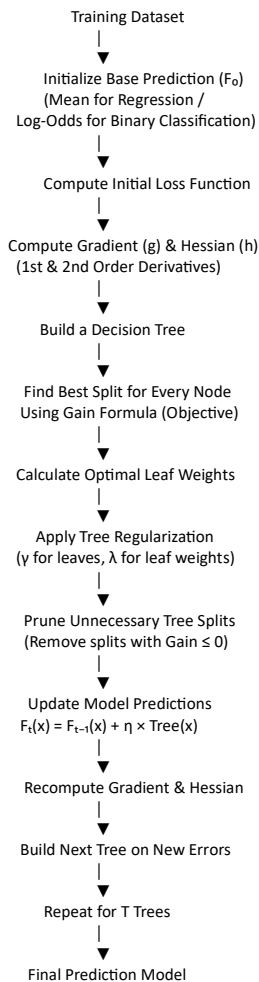
$$Obj = Loss + Regularization$$

This regularization term discourages overly complex trees.

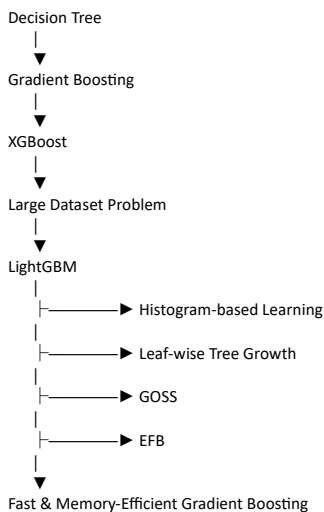
Important Terminologies

Term	Meaning
Booster	Ensemble of trees built sequentially
Objective Function	Quantity minimized during training
Regularization	Penalty to reduce overfitting
Sparse Data	Data with many missing or zero values
Tree Pruning	Removing splits that do not improve the objective sufficientl

Working Hierarchy :



LightGBM



Why Do We Need LightGBM?

XGBoost is already fast.
But imagine this dataset:

100 Million Rows
500 Features
Training XGBoost can become:

- Slow
- Memory intensive
- Expensive

Microsoft wanted a faster implementation.

The result was:

LightGBM (Light Gradient Boosting Machine)

. What is LightGBM?

Definition

LightGBM is an optimized implementation of Gradient Boosting that uses **histogram-based tree learning**, **leaf-wise tree growth**, and other optimization techniques to train much faster while using less memory.

Simple Definition

LightGBM is XGBoost optimized for **very large datasets**.

Innovation 1 — Histogram-Based Learning

Traditional algorithms examine many possible split points.

LightGBM first groups values into bins.

Instead of evaluating every value,
it evaluates the bins.

Visualization

Without histogram

18

19

20

21

22

23

24

Many candidate splits.

With histogram

18-20

21-23

24-26

Only a few candidate splits.

Benefits

- Faster split finding
- Lower memory
- Similar accuracy

Innovation 2 — Leaf-wise Tree Growth

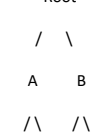
This is the most important interview topic.

XGBoost

Uses

Level-wise growth.

Root



C D E F

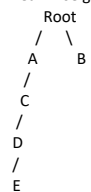
Every level grows together.

Balanced tree.

LightGBM

Uses

Leaf-wise growth.



Instead of growing every branch,

LightGBM grows **only the leaf that gives the highest reduction in loss**.

9. Advantages of LightGBM

- ✓ Extremely fast
- ✓ Lower memory usage
- ✓ Handles millions of rows
- ✓ High accuracy
- ✓ Native categorical feature support (when used appropriately)
- ✓ Parallel training

10. Disadvantages

- ✗ Can overfit on small datasets
- ✗ More sensitive to hyperparameters than Random Forest
- ✗ Leaf-wise trees may become very deep without constraints

11. XGBoost vs LightGBM

XGBoost	LightGBM
Level-wise tree growth	Leaf-wise tree growth
Exact/Histogram split methods	Histogram-based splits
Generally slower on very large datasets	Usually faster on large datasets
Higher memory usage	Lower memory usage
Often more stable on small datasets	Can overfit on small datasets

Putting Everything Together

Dataset
↓
Histogram Binning
↓
Compute Gradients
↓
GOSS
(Remove Less Important Rows)
↓
EFB
(Merge Exclusive Features)
↓
Leaf-wise Growth
↓
Prediction

8. Advantages
- ✓ Very fast
 - ✓ Low memory usage
 - ✓ Excellent for large datasets
 - ✓ Handles missing values
 - ✓ Supports categorical features
 - ✓ High accuracy

9. Disadvantages
- ✗ Can overfit small datasets
 - ✗ Sensitive to hyperparameters
 - ✗ Less interpretable than a single tree

Decision Tree vs Random Forest vs Gradient Boosting vs XGBoost vs LightGBM					
Feature	Decision Tree	Random Forest	Gradient Boosting	XGBoost	LightGBM
Single Tree	✓	✗	✗	✗	✗
Ensemble	✗	✓	✓	✓	✓
Bagging	✗	✓	✗	✗	✗
Boosting	✗	✗	✓	✓	✓
Regularization	✗	Limited	Limited	✓	✓
Missing Value Handling	Depends on implementation	Depends	Limited	✓	✓
Histogram Learning	✗	✗	✗	Optional	✓
GOSS	✗	✗	✗	✗	✓
EFB	✗	✗	✗	✗	✓
Tree Growth	Standard	Standard	Standard	Level-wise	Leaf-wise
Speed	Fast	Moderate	Slow	Fast	Very Fast

Which Algorithm Should You Choose?

- Small Dataset (<10,000 rows)
- ✓ Decision Tree (baseline)
 - ✓ Random Forest
- Reason
- Less tuning
 - Lower risk of overfitting

- Medium Dataset (10K–100K rows)
- ✓ Random Forest
 - ✓ XGBoost
- Choose:
- **Random Forest** if you need a strong baseline with minimal tuning.
 - **XGBoost** if you're optimizing for predictive performance and are willing to tune hyperparameters.

- Large Dataset (>100K rows)
- ✓ LightGBM
- Reason
- Histogram algorithm
 - GOSS
 - EFB
 - Fast training

Maximum Accuracy

Usually compare

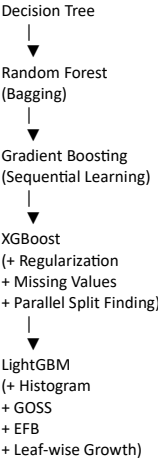
- XGBoost
- LightGBM
- CatBoost

using cross-validation.
There is **no universal winner**—the best choice depends on the dataset.

12. Hyperparameter Cheat Sheet

Parameter	Purpose
num_leaves	Maximum leaves
learning_rate	Step size
n_estimators	Number of trees
feature_fraction	Feature sampling
bagging_fraction	Row sampling
min_data_in_leaf	Minimum samples in a leaf
max_depth	Maximum tree depth
lambda_l1	L1 regularization
lambda_l2	L2 regularization

Complete Boosting Evolution



CatBoost

CatBoost (Categorical Boosting), developed by **Yandex**. It is one of the most important boosting algorithms because it solves one of the biggest problems in Machine Learning:

. **Why Do We Need CatBoost?**
Suppose we have this dataset.

Age	Gender	City	Salary
25	Male	Delhi	50K
30	Female	Mumbai	70K
28	Male	Bhopal	60K

Notice
Gender
City
These are **categorical features**.
Most ML algorithms cannot directly understand text.
They only understand numbers.

2. Traditional Solution

We convert
Male

Female
into
0

1
This is called
Label Encoding

Or
Male

Female
becomes
Male Female

1 0
0 1
This is called
One-Hot Encoding.

Problem with this? : 100 Features -> 100 columns
Solution : CATBoost

4. What is CatBoost?

Definition

CatBoost (**Categorical Boosting**) is a gradient boosting algorithm that **handles categorical features natively** and reduces prediction bias using **Ordered Boosting**.
CatBoost directly Accepts all the categoris without any encoding process.

Eg: cat_features = ['Gender', 'City']

Main Innovations

CatBoost has **three major innovations**.

Categorical Features

↓

Target Statistics

↓

Ordered Boosting

↓

Symmetric Trees

↓

Prediction

These three ideas make CatBoost unique.

7. Innovation 1 — Native Categorical Handling

Instead of

City

↓

One-Hot Encoding

CatBoost internally converts categories into meaningful numerical statistics while training.

You only specify

cat_features

Everything else

is automatic.

Example

Dataset

City **Bought**

Delhi Yes

Delhi No

Mumbai Yes

Bhopal Yes

Instead of

creating

three columns,

CatBoost learns a useful numeric representation for each category during training.

8. Innovation 2 — Ordered Boosting

This is CatBoost's biggest contribution.

To understand it,

we first need to understand **prediction leakage**.

Suppose

Student **Marks**

A 95

B 40

C 80

If we calculate statistics for Student A using **Student A's own target**, we've accidentally used

future information.

This is called

Target Leakage.

It leads to overly optimistic training.

CatBoost solves this by using

Ordered Boosting.

Instead of using all data, it uses only previously seen data

when calculating statistics.

No future information leaks.

Think of it like this:

Imagine students standing in a queue.

A

↓

B

↓

C

↓

D

When processing

Student C, CatBoost only uses A and B. It never uses C or D. This prevents leakage.

9. Innovation 3 — Symmetric Trees

Decision Trees

look like



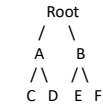
Branches Have different depths. These are **asymmetric trees**.

CatBoost

uses

Symmetric (Oblivious) Trees.

Example



Every level
splits using
the same feature
and threshold.

Advantages

- Faster prediction
- Better CPU efficiency
- Simpler tree structure

Disadvantage

Slightly less flexible
than arbitrary trees.

10. Why Does CatBoost Work Well?

Because it solves

three major problems simultaneously.

- ✓ Handles categorical features.
- ✓ Prevents target leakage.
- ✓ Uses fast symmetric trees.

11. Applications

CatBoost performs well in

- Banking
- Insurance
- E-commerce
- Recommendation Systems
- Customer Churn Prediction
- Credit Scoring
- Marketing Analytics

Any dataset with many categorical columns.

12. XGBoost vs LightGBM vs CatBoost

Feature	XGBoost	LightGBM	CatBoost
Native categorical features	✗	Limited	✓
One-Hot Encoding required	Usually	Sometimes	No
Ordered Boosting	✗	✗	✓
Symmetric Trees	✗	✗	✓
Large datasets	✓	✓	✓
Many categorical features	Good	Good	Excellent

Complexity

Training

Approximately

$$O(T \cdot n \cdot d)$$

Prediction

$$O(T \log n)$$

Like XGBoost and LightGBM, practical speed depends heavily on implementation details and tree depth.

Important Terminologies

Term	Meaning
Categorical Feature	Feature with discrete categories (e.g., City, Gender)
Ordered Boosting	Uses only earlier samples to compute target statistics and train without leakage
Target Leakage	Accidentally using future or target information during training
Symmetric Tree	Every level uses the same split across all nodes

Memory Trick

CatBoost

↓

Cat = Categorical Features

Boost = Gradient Boosting

Or remember:

CatBoost =

Categories

+

Ordered Boosting

+

Symmetric Trees

Things to Remember Forever

- **CatBoost is a gradient boosting algorithm optimized for categorical features.**
- It avoids manual one-hot encoding by learning useful numeric representations of categories internally.

- **Ordered Boosting** prevents target leakage by using only previously seen examples when computing target statistics.
- **Symmetric (oblivious) trees** make prediction fast and efficient.

CatBoost's Solution — Ordered Target Statistics

Instead of using
all rows,
CatBoost creates
a random order.

Example
Original

A
B
C
D
E

Random permutation

C
A
E
B
D

Now compute
statistics
sequentially.

Suppose

Current row

E

CatBoost uses
only

C

A

Never

B

D

because
they occur later.

Even

E itself
is not included.

No leakage.

9. Numerical Example

Random order

Order	City	Bought
1	Delhi	1
2	Mumbai	0
3	Delhi	1
4	Delhi	0
5	Mumbai	1

Row 1

Delhi

No previous Delhi.

Use prior value.

Suppose

Prior

0.5

Encoding

0.5

Row 2

Mumbai

No previous Mumbai.

Again

0.5

Row 3

Delhi

Previous Delhi

1

Average = 1

Encoding

1

Row 4

Delhi

Previous Delhi values

1

1

Average = 1

Encoding

1

Notice

Current target (0)

was NOT used.

Row 5

Mumbai

Previous Mumbai

0

Average = 0

Again

Current row

ignored.

Final Encodings

City Encoding

Delhi 0.5

Mumbai 0.5

Delhi 1

Delhi 1

Mumbai 0

No leakage.

10. Why Prior Value?

Suppose

first occurrence

of

Delhi.

There are

no previous observations.

Average

cannot be computed.

CatBoost starts with

a prior

(often related to the global target mean)

Example

Prior = 0.5

Later,

real observations

gradually replace

the prior.

11. Ordered Boosting

Now comes the second innovation.

Gradient Boosting

builds tree

using

entire dataset.

CatBoost

uses

ordered statistics

at every iteration.

Pipeline

Random Permutation

↓

Ordered Statistics

↓

Train Tree

↓

Next Iteration

↓

New Ordered Statistics

↓

Next Tree

Thus,

every boosting iteration

avoids leakage.

12. Complete Training Flow

Dataset

↓

Random Permutation

↓

Ordered Target Statistics

↓

Convert Categories

↓

Compute Gradient

↓

Build Symmetric Tree

↓

Update Prediction

↓

Repeat

13. Why Is CatBoost More Accurate?

Without CatBoost

Category

↓

One-Hot Encoding

↓

Possible Leakage

↓

Training

With CatBoost

Category

↓

Ordered Statistics

↓
No Leakage
↓

Training

The model learns from past information only, making its training process closer to what it will experience on unseen data.

Support Vector Machine (SVM)

Why Do We Need SVM?

Suppose we want to classify emails as Spam or Not Spam.

Dataset:

Email Length	Contains "Free"	Label
Short	Yes	Spam
Long	No	Not Spam
Medium	Yes	Spam

A classifier has to separate these two classes.

Many algorithms can do this:

- Logistic Regression
- Decision Tree
- Random Forest
- XGBoost

So why invent SVM?

Because **many separating lines are possible**, but **not all are equally good**.

Problem with Ordinary Linear Classification

For two classes, separator line can be many and at many positions or angle so which to choose it problem

o o o o o class A
----- Separator 1
----- Sepearator 2

x x x x x class B

SVM's Answer

SVM says:

Don't choose **any** separating line.

Choose the one with the **largest margin**.

What is Margin?

Margin = Distance between the decision boundary and the nearest training points.

Visualization

o o o o o

Margin

=====

Decision Boundary

Margin

x x x x x

The empty space around the decision boundary is called the **margin**.

Why Larger Margin?

Suppose two models.

Small Margin

o o o

=====

x

x x

A tiny amount of noise can cause misclassification.

Large Margin

o o o

=====

x x x

Even if the data shifts slightly,
the classifier still works.

Imagine driving on a road.

Narrow Road

| Car |

One small mistake

↓

Accident.

Wide Road

| Car |

Much safer.

The **wide road** is exactly what SVM tries to achieve.

Support Vector Machine (SVM)

Support Vector Machine is a supervised learning algorithm that finds the hyperplane that maximizes the margin between different classes.

Simple Definition

SVM finds the **best separating boundary** by maximizing the distance to the nearest data points.

7. Decision Boundary

A decision boundary is the line (or plane) that separates different classes.

Example

o o o o

x x x x

The line is the **decision boundary**.

8. What is a Hyperplane?

In Machine Learning,

the separator is called a **hyperplane**.

2D

A line

o o o

x x x

3D

A plane

Imagine a flat sheet dividing space into two halves.

Higher Dimensions

A hyperplane is the generalization of a line (2D) or plane (3D) to any number of dimensions.

Hyperplane Equation

Exactly like Linear Regression,

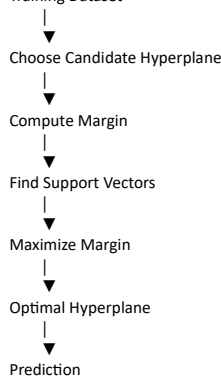
$$w^T x + b = 0$$

where

- w = weight vector
- x = feature vector
- b = bias

SVM Training Flow

Training Dataset



Applications

SVM has been widely used for:

- Face Detection
- Text Classification
- Spam Detection
- Handwriting Recognition
- Bioinformatics
- Medical Diagnosis

Although deep learning has replaced SVM in many large-scale vision tasks, SVM is still a strong choice for many **small to medium-sized datasets**.

Advantages

- ✓ Works well in high-dimensional spaces
- ✓ Effective with small and medium datasets
- ✓ Can model nonlinear boundaries using kernels (covered later)
- ✓ Robust because it maximizes the margin

14. Disadvantages

- ✗ Training becomes slow on very large datasets
- ✗ Sensitive to hyperparameter tuning
- ✗ Less interpretable than simple linear models

Important Terminologies

Term	Meaning
Hyperplane	Decision boundary in any dimension

Term	Meaning
Margin	Distance from boundary to nearest samples
Support Vectors	Training points closest to the boundary
Maximum Margin	Largest possible separation between classes

Formula Sheet Hyperplane

$$w^T x + b = 0$$

Prediction

$$\hat{y} = \text{sign}(w^T x + b)$$

Distance

$$\text{Distance} = \frac{|w^T x + b|}{\|w\|}$$

Functional Margin

$$\gamma_i = y_i(w^T x_i + b)$$

Geometric Margin

$$\hat{\gamma}_i = \frac{y_i(w^T x_i + b)}{\|w\|}$$

Margin

$$\text{Margin} = \frac{2}{\|w\|}$$

Optimization

$$\min \frac{1}{2} \|w\|^2$$

Subject to

$$y_i(w^T x_i + b) \geq 1$$

Functional Margin vs Geometric Margin

Functional Margin	Geometric Margin
$y_i(w^T x_i + b)$	$\frac{y_i(w^T x_i + b)}{\ w\ }$
Depends on scale	Scale invariant
Raw confidence score	True perpendicular distance
Not suitable for optimization	Used in SVM optimization

Why do we need a Kernel?

A **Linear SVM** can only separate data using a **straight line (or hyperplane)**.

Example:

○ ○ ○ ○

× × × ×

A straight line works.

But suppose the data looks like this:

○ ○ ○

○ ○

×

○ ○

○ ○ ○

No straight line can separate them.

Linear SVM fails.

So we need a way to separate **non-linear data**.

Solution

Transform the data into a **higher-dimensional space** where it becomes linearly separable.

Example:

Original Feature

x

Transform into

x²

or

(x₁, x₂, x₁², x₂², x₁x₂)

Now a straight line can separate the data.

Problem

If we explicitly transform every feature,

100 features

↓

may become

10,000+ features.

Very slow and memory-intensive.

Kernel Trick

Instead of actually creating those thousands of new features, the **Kernel Trick** computes the result **as if the transformation had been done**, without explicitly performing it.

Interview Definition

A **Kernel** is a function that computes the similarity between two data points in a higher-dimensional feature space without explicitly transforming the data.

This makes SVM efficient while still learning nonlinear boundaries.

Formula (Know It)

$$K(x, z) = \phi(x)^T \phi(z)$$

Don't worry about deriving it.

Just know:

- $\phi(x)$ = transformed features
- Kernel computes their dot product directly.

Types of Kernels

1. Linear Kernel

Formula

$$K(x, z) = x^T z$$

Uses the original features.

Use when

- Data is nearly linear.
- Text classification (high-dimensional sparse data).
- Very large datasets.

2. Polynomial Kernel

Formula

$$K(x, z) = (x^T z + c)^d$$

Creates polynomial decision boundaries.

Use when

- Relationship follows polynomial patterns.

Not as commonly used today.

3. RBF (Gaussian) Kernel 🌟 Most Important

Formula

$$K(x, z) = e^{-\gamma \|x - z\|^2}$$

This is the **default kernel** for many nonlinear problems.

Intuition

- Nearby points → very similar.
- Far-away points → less similar.

Use when

- Data is nonlinear.
- You don't know the relationship.

This is the kernel you'll use most often.

Gamma (γ)

Very common interview question.

Gamma controls **how much influence a single training point has**.

Large Gamma

- Only nearby points influence the boundary.
- Complex boundary.
- Higher chance of overfitting.

Small Gamma

- Even far-away points influence the boundary.
- Smooth boundary.
- Higher chance of underfitting.

Easy way to remember

Large γ → Complex model → Overfitting

Small γ → Smooth model → Underfitting

Which Kernel Should You Use?

Kernel	When to Use
Linear	Linear data, text classification, large datasets
Polynomial	Polynomial relationships
RBF	General nonlinear problems (default choice)

Naive Bayes

Why Do We Need Naive Bayes?

Suppose you receive an email.

"Congratulations!!

You won ₹10,00,000"

Question:

Is it

- Spam
- Not Spam

How do we decide?

One approach is to check

- Does it contain "Win"?
- Does it contain "Free"?
- Does it contain "Offer"?
- Does it contain "Congratulations"?

Instead of manually creating rules,
Naive Bayes calculates the probability that the email belongs to each class.
Then it chooses the class with the highest probability.

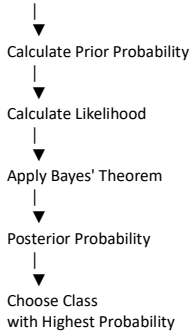
Memory Trick

Think of Bayes' theorem as:

$$\text{Posterior} = \frac{\text{Likelihood} \times \text{Prior}}{\text{Evidence}}$$

Working of Naive Bayes

Training Data



Advantages

- ✓ Very fast to train
- ✓ Works well on text data
- ✓ Requires relatively little training data
- ✓ Easy to implement

Disadvantages

- ✗ Independence assumption is often unrealistic
- ✗ Performance may be lower when features are highly dependent
- ✗ Posterior probabilities may not be well calibrate

Gaussian Naive Bayes ★★★★★

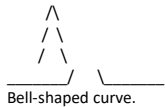
Used for
Continuous Numerical Features
Examples

- Height
- Weight
- Salary
- Age
- Temperature

Assumption

Each feature follows a **Normal (Gaussian) Distribution**.

Visualization



Bell-shaped curve.

Probability Formula

$$P(x | C) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

For interviews, you **don't need to derive this formula**—just know it models continuous features using a Gaussian distribution.

Use Cases

- Medical diagnosis
- Sensor readings
- Income prediction
- Continuous measurements

9. Multinomial Naive Bayes ★★★★★

Used for
Count Data
Examples
Free appears 4 times

Offer appears 2 times

Win appears 1 time
The **number of occurrences** matters.

Applications

- Spam Detection
- Document Classification
- Sentiment Analysis
- News Classification

This is the **most commonly used Naive Bayes variant for NLP**.

10. Bernoulli Naive Bayes ★★★★★

Used for
Binary Features
Each feature is either
Present

or

Absent
Example
Email
Contains "Free"?

Yes
↓
1
Contains "Offer"?

No
↓
0
Only presence/absence matters,
not how many times the word appears.

11. Difference Between Multinomial & Bernoulli

Email
Free Free Free Offer

Multinomial

Counts frequency.
Free = 3

Offer = 1

Bernoulli

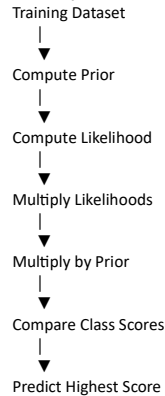
Only checks whether the word exists.
Free = Yes

Offer = Yes

12. Which Variant Should I Use?

Feature Type	Algorithm
Continuous numerical values	Gaussian NB
Word counts / frequencies	Multinomial NB
Binary (Yes/No, 0/1)	Bernoulli NB

13. Complete Working



Formula Sheet

Bayes Theorem

$$P(C | X) = \frac{P(X | C)P(C)}{P(X)}$$

Naive Assumption

$$P(X | C) = \prod P(x_i | C)$$

Final Formula

$$P(C | X) \propto P(C) \prod P(x_i | C)$$

The symbol $\propto$ means "**proportional to**", indicating we've ignored the constant denominator $P(X)$.

Complexity

Suppose

- n = samples
- d = features

Training

$$O(nd)$$

Prediction (per sample)

$$O(d \times \text{number of classes})$$

Naive Bayes is extremely fast because it only computes simple probabilities.

Advantages

- ✓ Very fast
- ✓ Works well with small datasets
- ✓ Excellent for text classification
- ✓ Easy to implement

Disadvantages

- ✗ Assumes conditional independence
- ✗ Can perform poorly if features are strongly dependent
- ✗ Zero probabilities can be a problem (we'll fix this with Laplace Smoothing next)

K-Nearest Neighbors

KNN is a Lazy Learning Algorithm ★★★★★

One of the most common interview questions.

Unlike Logistic Regression,
Decision Tree,
SVM,
Naive Bayes,
KNN

does **not** learn during training.

Training phase

↓

Simply stores the dataset.

Learning happens

only when

a prediction is requested.

Advantages

- ✓ Very simple
- ✓ No training required
- ✓ Naturally handles nonlinear decision boundaries
- ✓ Easy to understand and implement

12. Disadvantages

- ✗ Slow prediction
- ✗ Stores entire dataset
- ✗ Sensitive to irrelevant features
- ✗ Requires feature scaling
- ✗ Performance decreases in very high dimensions (Curse of Dimensionality, covered later)

KNN vs Other Algorithms

Algorithm	Training	Prediction
Logistic Regression	Slow	Fast
Decision Tree	Moderate	Fast
Random Forest	Slow	Fast
SVM	Slow	Fast
KNN	Very Fast	Slow

K-Nearest Neighbors (KNN) is a supervised, non-parametric, instance-based learning algorithm used for both classification and regression. During training, it simply stores the training data instead of learning a mathematical model, which is why it is called a lazy learner. During prediction, it computes the distance between the new sample and all training samples, selects the K nearest neighbors, and predicts the class using majority voting for classification or the average value for regression.

Curse of Dimensionality ★★★★★

One of the most frequently asked KNN interview questions.

What is it?

As the **number of features (dimensions)** increases, the concept of "nearest" becomes less meaningful.

In high dimensions:

- Almost every point is far away.
- Distances between points become very similar.
- KNN struggles to distinguish close neighbors from distant ones.

Solution

Reduce dimensions using techniques such as:

- PCA (Principal Component Analysis)
- Feature Selection
- Autoencoders (Deep Learning)

We'll study PCA in the Unsupervised Learning module.

What is Unsupervised Learning?

Definition

Unsupervised Learning is a type of Machine Learning where the model learns patterns, structures, or relationships from unlabeled data.

Unlike supervised learning,

we **do not know the correct output**.

Types of Unsupervised Learning

There are four major categories.

Unsupervised Learning

- └─ Clustering
- └─ Dimensionality Reduction
- └─ Association Rule Learning
- └─ Anomaly Detection

A. Clustering ★★★★★

Groups similar data points together.

Example:

Customer Segmentation

Customers

↓

Group 1 → Students

Group 2 → Working Professionals

Group 3 → Senior Citizens

Algorithms

- K-Means
- Hierarchical Clustering
- DBSCAN
- Gaussian Mixture Models

B. Dimensionality Reduction ★★★★★★

Suppose a dataset has

500 features.

Many may be redundant.

Dimensionality Reduction compresses the data while preserving as much useful information as possible.

Example

500 Features

↓

20 Features

Algorithms

- PCA
- t-SNE
- UMAP

Benefits

- Faster training
- Better visualization
- Less overfitting
- Lower storage requirements

C. Association Rule Learning ★★★★★★

Finds relationships between items.

Example

Shopping Basket

Milk

Bread

Butter

Rule

Milk

↓

Bread

Meaning

Customers buying milk often buy bread too.

Algorithms

- Apriori
- FP-Growth

Applications

- Product recommendations
- Market basket analysis

D. Anomaly Detection ★★★★★★

Finds unusual observations.

Example

Normal transactions

₹500

₹800

₹1000

Suddenly

₹8,00,000

Possible fraud.

Applications

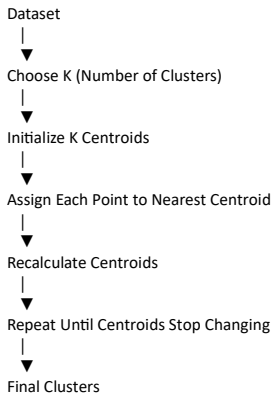
- Fraud Detection
- Cybersecurity

- Manufacturing defects
- Network intrusion detection

4. What is Clustering?

Definition

Clustering is the process of grouping similar data points into clusters so that points within the same cluster are more similar to each other than to points in other clusters.



K-Means is an unsupervised clustering algorithm that partitions data into K clusters by minimizing the distance between data points and their assigned cluster centroid.

Advantages

- ✓ Easy to understand
- ✓ Fast on medium-sized datasets
- ✓ Simple implementation
- ✓ Works well when clusters are compact and roughly spherical

12. Disadvantages

- ✗ Must choose K beforehand
- ✗ Sensitive to initialization
- ✗ Sensitive to outliers
- ✗ Assumes spherical clusters
- ✗ Performs poorly when clusters have different densities or irregular shapes

What is WCSS?

WCSS stands for

Within Cluster Sum of Squares

Also called

- Inertia (Scikit-learn)
- SSE (Sum of Squared Errors)

Definition

It is the **sum of squared distances of every point from its assigned centroid.**

Good clusters

↓
Small distance
↓
Small WCSS

3. Formula

Suppose

- K clusters
- C_i = Cluster i
- μ_i = Centroid of Cluster i

Then

$$WCSS = \sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|^2$$

Meaning

For every cluster

↓
For every point

↓
Compute squared distance from centroid
↓
Add everything.

Silhouette Score ★★★★★

This is generally **more informative** than the Elbow Method because it evaluates both:

- Cluster compactness
- Cluster separation

Intuition

A good cluster should satisfy:

- Points are **close to their own cluster.**
- Points are **far from other clusters.**

Silhouette Score measures exactly this.

10. Formula

For one point

Let

- a = Average distance to points in the **same cluster**
- b = Average distance to points in the **nearest different cluster**

Then

$$S = \frac{b - a}{\max(a, b)}$$

11. Meaning of a and b

a

Average distance to your own cluster.

Smaller

↓
Better.

b

Average distance to the nearest neighboring cluster.

Larger

↓
Better.

12. Range

Silhouette Score

$$-1 \leq S \leq 1$$

$S = 1$

Excellent clustering.

Point is well inside its cluster.

$S = 0$

Point lies near the boundary between two clusters.

$S < 0$

Bad clustering.

Point is probably assigned to the wrong cluster.

Visual Intuition

Good

●●●●●

▲▲▲▲▲

Large separation.

Score close to 1.

Average

●●●▲▲▲

Mixed clusters.

Score around 0.

Bad

●▲●▲●▲

Poor grouping.

Negative score.

13. Example

Suppose

For one point

$$\begin{aligned} a &= 2 \\ b &= 8 \end{aligned}$$

Score

$$= \frac{8 - 2}{8} = 0.75$$

Very good.

Suppose

$$\begin{aligned} a &= 5 \\ b &= 4 \end{aligned}$$

Score

$$= \frac{4 - 5}{5} = -0.2$$

Likely misclassified.

14. Elbow vs Silhouette

Elbow Method	Silhouette Score
Uses WCSS	Uses distances within and between clusters
Visual inspection	Numerical score
Can be subjective	More objective
Faster	Slightly more computationally expensive

15. Complexity

Suppose

- n = samples
- k = clusters

- d = features
- I = iterations

K-Means $O(nkdI)$

Elbow Method
Runs K-Means multiple times.
If tested for m values of K ,

$O(mnkdI)$

Silhouette Score
Naively, computing pairwise distances is

$O(n^2)$

though optimized implementations can reduce practical runtime.

Formula Sheet

WCSS

$$WCSS = \sum ||x - \mu||^2$$

Objective

$$\min(WCSS)$$

Silhouette Score

$$S = \frac{b - a}{\max(a, b)}$$

Range

$$-1 \leq S \leq 1$$

Why Do We Need Hierarchical Clustering?

Suppose we have customer data:

With K-Means,
we must first choose
 $K = ?$

But what if we **don't know** how many clusters exist?
Hierarchical Clustering solves this by creating a **hierarchy of clusters**, allowing us to decide the number of clusters later.

What is Hierarchical Clustering?

Definition

Hierarchical Clustering is an unsupervised learning algorithm that builds a hierarchy (tree) of clusters by repeatedly merging or splitting clusters based on similarity.

Unlike K-Means,
it doesn't directly produce K clusters.
Instead,
it creates a tree called a **Dendrogram**.

Types of Hierarchical Clustering

There are two approaches.
Hierarchical Clustering

└─ Agglomerative
└─ Divisive

5. Agglomerative Clustering ★★★★★

This is the **most commonly used** approach.

Also called

Bottom-Up Clustering

Working

Initially,
every point is its own cluster.

Divisive Clustering ★★★★★★

Much less common.

Also called

Top-Down Clustering

Instead of merging,

it starts with

one large cluster.

ABCD

↓

Split

Does Hierarchical Clustering Need K?

No.

This is its biggest advantage.

Instead,

build the hierarchy first.

Then,

choose the number of clusters

by cutting the dendrogram.

Time Complexity

Suppose

- n = number of samples

Naive implementation

$O(n^3)$

Modern optimized implementations (such as many agglomerative algorithms)

$O(n^2)$

Still much slower than K-Means for large datasets.

What is Linkage?

Definition

Linkage defines how the distance between two clusters is computed during Hierarchical Clustering.

Different linkage methods can produce **different cluster structures** from the same dataset.

3. Types of Linkage

There are four major linkage methods:

Hierarchical Clustering

└─ Single Linkage
└─ Complete Linkage
└─ Average Linkage
└─ Ward Linkage

Single Linkage (Nearest Neighbor)

Definition

The distance between two clusters is the **minimum distance** between any pair of points from the two clusters.

Formula

$$D(A, B) = \min_{x \in A, y \in B} d(x, y)$$

Complete Linkage (Farthest Neighbor)

Definition

Distance between two clusters is the **maximum distance** between any pair of points.

Formula

$$D(A, B) = \max_{x \in A, y \in B} d(x, y)$$

Average Linkage

Instead of minimum or maximum,
take the **average** of all pairwise distances.

Formula

$$D(A, B) = \frac{1}{|A| |B|} \sum_{x \in A} \sum_{y \in B} d(x, y)$$

Comparison of Linkage Methods

Linkage	Distance Used	Best For	Weakness
Single	Minimum	Arbitrary shapes	Chaining effect
Complete	Maximum	Compact clusters	Sensitive to outliers
Average	Mean distance	Balanced performance	Slightly slower
Ward	Increase in variance	Compact, spherical clusters	Requires Euclidean distance

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN is one of the most important clustering algorithms in Machine Learning.

Unlike **K-Means**, DBSCAN **does not try to divide data into K clusters**.

Instead, it asks a simple question:

"Which points are packed closely together?"

Points that are densely packed form a cluster.

Points that are isolated are treated as **noise (outliers)**.

Real Intuition

Imagine you're looking at people in a large park.

You notice:

👤 👤 👤 👤 👤

(Group 1)

👤 👤 👤 👤

(Group 2)

👤

(Alone)

You would naturally say:

- These people form one group.
- Those people form another group.
- The person standing alone isn't part of any group.

DBSCAN works exactly like this.

It **looks for dense groups of points** and labels isolated points as noise.

Why Do We Need DBSCAN?

Suppose your data looks like this:

There are clearly two clusters.

K-Means can find them.

Now consider:

* *

* *

This is a ring-shaped cluster.

K-Means will incorrectly split it because it assumes clusters are roughly spherical.
DBSCAN has **no such assumption** and can discover clusters of arbitrary shapes.

Core Idea

DBSCAN says:

If **many points are close together, they belong to the same cluster**.

It uses **density**, not distance to a centroid.

Two Important Parameters

DBSCAN only needs two parameters.

1. Epsilon (ϵ)

Pronounced **epsilon**.

It defines the **radius** around each point.

Think of drawing a circle around every point.



(Circle Radius = ϵ)

Only points inside this circle are considered neighbors.

2. MinPts (Minimum Points)

Minimum number of neighboring points required to form a dense region.

Example:

MinPts = 5

If a point has at least **5 neighbors within ϵ** , it is considered a dense area.

Types of Points

DBSCAN classifies every point into one of three categories.

1. Core Point ★★★★★

A point is a **core point** if it has at least **MinPts neighbors** within the ϵ -radius.

Example:



Suppose

ϵ includes all six surrounding points.

MinPts = 5

Since X has six neighbors,

X is a **Core Point**.

2. Border Point

A border point does **not** have enough neighbors to be a core point.

However,

it lies within the ϵ -radius of a **core point**.

Core Point

X



The small point has only one nearby neighbor.

It is not dense enough to be a core point.

But since it touches a core point,

it becomes a **Border Point**.

3. Noise Point (Outlier)

If a point:

- Is not a core point

- Is not reachable from any core point

then it is classified as **Noise**.

Cluster



(Noise)

How DBSCAN Works

Imagine these points:



Step 1

Pick any point.

Step 2

Draw an ϵ -radius circle around it.

Step 3

Count neighbors.

If neighbors $\geq$ MinPts



It becomes a Core Point.

Step 4

Expand the cluster.

Every neighboring core point is added.

Then their neighbors are explored too.

The cluster keeps growing until no more connected dense points exist.

Step 5

Pick another unvisited point.

Repeat.

Step 6

Any remaining isolated points become noise.

Visualization

Cluster 1



Cluster 2



Noise



Important Concept: Density Reachability

Suppose

A ---- B ---- C ---- D

A is close to B.

B is close to C.

C is close to D.

Even if

A is **not directly close to D**,

DBSCAN still puts them in the same cluster because they are connected through a chain of dense regions.

This idea is called **density reachability**.

Advantages

1. No Need to Choose K

Unlike K-Means,

you don't need to specify the number of clusters beforehand.

2. Finds Arbitrary Shapes

Works well for:

- Circles
- Spirals
- Curved shapes
- Irregular clusters

3. Detects Outliers Automatically

Noise points are identified without any extra algorithm.

4. Robust to Outliers

Outliers do not significantly influence the discovered clusters.

Disadvantages

1. Choosing ϵ Can Be Difficult

If ϵ is too small:

- Many points become noise.

If ϵ is too large:

- Separate clusters may merge.

2. Struggles with Varying Densities

If one cluster is very dense and another is sparse, a single ϵ value may not work well for both.

3. Performance in High Dimensions

Distance becomes less meaningful as the number of features grows (the "curse of dimensionality"), so DBSCAN may become less effective without preprocessing or dimensionality reduction.

DBSCAN vs K-Means

Feature	K-Means	DBSCAN
Need K?	Yes	No
Cluster Shape	Mostly spherical	Arbitrary shapes
Finds Outliers	No	Yes
Uses Centroids	Yes	No
Uses Density	No	Yes
Sensitive to Outliers	Yes	Less
Handles Noise	Poorly	Well

Hyperparameters

Parameter Meaning

ϵ (eps) Radius used to search for neighboring points

Parameter	Meaning
MinPts	Minimum number of neighbors required to form a dense region

Choosing ϵ

A common approach is the **k-distance plot**:

1. For each point, compute the distance to its k -th nearest neighbor (often using $k = \text{MinPts} - 1$).
2. Sort these distances.
3. Look for the "elbow" (a sharp change in the curve).
4. Choose ϵ near that elbow.

Python Example

```
from sklearn.cluster import DBSCAN
```

```
db = DBSCAN(
    eps=0.5,
    min_samples=5
)
```

```
labels = db.fit_predict(X)
```

Here:

- $\text{eps}=0.5 \rightarrow$ Neighborhood radius.
- $\text{min_samples}=5 \rightarrow$ Minimum neighbors to form a dense region.
- labels contains cluster IDs, and -1 indicates noise points.

Common Interview Questions

1. Why is DBSCAN better than K-Means?

Because it:

- Doesn't require specifying K.
- Can find irregularly shaped clusters.
- Automatically detects outliers.

2. When should you use DBSCAN?

Use DBSCAN when:

- The number of clusters is unknown.
- You expect noise or outliers.
- Clusters may have irregular shapes.
- Data forms dense regions.

3. When should you avoid DBSCAN?

Avoid it when:

- Clusters have very different densities.
- The data has many dimensions.
- Choosing a single ϵ value is difficult.

4. What is the difference between a Core Point and a Border Point?

- **Core Point:** Has at least MinPts neighbors within ϵ .
- **Border Point:** Does not meet the neighbor requirement itself but lies within the ϵ -neighborhood of a core point.

Machine Learning Coding Cheat Sheet (Scikit-learn)

0. Common Imports

```
import numpy as np
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import (
    StandardScaler,
    MinMaxScaler,
    LabelEncoder,
    OneHotEncoder
)
```

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    mean_squared_error,
    mean_absolute_error,
    r2_score,
    silhouette_score
)
```

1. Linear Regression

Import

```
from sklearn.linear_model import LinearRegression
```

Model

```
model = LinearRegression()
```

Important Hyperparameters

```
fit_intercept=True
copy_X=True
positive=False
```

Train

```
model.fit(X_train, y_train)
```

Predict

```
y_pred = model.predict(X_test)
```

Important Attributes

```
model.coef_
```

```
model.intercept_
```

2. Ridge Regression

```
from sklearn.linear_model import Ridge
```

```
model = Ridge(
    alpha=1.0,
    fit_intercept=True
)
```

Important

```
alpha
```

3. Lasso Regression

```
from sklearn.linear_model import Lasso
```

```
model = Lasso(
    alpha=0.1
)
```

Important

```
alpha
```

4. ElasticNet

```
from sklearn.linear_model import ElasticNet
```

```
model = ElasticNet(
    alpha=1,
    l1_ratio=0.5
)
```

Important

```
alpha
```

```
l1_ratio
```

5. Logistic Regression

Import

```
from sklearn.linear_model import LogisticRegression
```

Model

```
model = LogisticRegression(
    penalty='l2',
    C=1.0,
    solver='lbfgs',
    max_iter=100
)
```

Important Hyperparameters

```
penalty
```

```
C
```

```
solver
```

```
max_iter
```

6. Decision Tree

```
from sklearn.tree import DecisionTreeClassifier
```

```
model = DecisionTreeClassifier(
    criterion='gini',
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    random_state=42
)
```

Important

```
criterion
```

```
max_depth
```

```
min_samples_split
```

```
min_samples_leaf
```

```
random_state
```

7. Random Forest

```
from sklearn.ensemble import RandomForestClassifier
```

```
model = RandomForestClassifier(
    n_estimators=100,
    criterion='gini',
    max_depth=None,
    max_features='sqrt',
    random_state=42
)
```

Important

```
n_estimators
```

```
max_depth
```

```
max_features
```

```
criterion
```

```
random_state
```

8. AdaBoost

```
from sklearn.ensemble import AdaBoostClassifier
```

```
model = AdaBoostClassifier(  
    n_estimators=100,  
    learning_rate=1.0,  
    random_state=42  
)  
Important  
n_estimators
```

learning_rate

9. Gradient Boosting

```
from sklearn.ensemble import GradientBoostingClassifier  
model = GradientBoostingClassifier(  
    n_estimators=100,  
    learning_rate=0.1,  
    max_depth=3,  
    random_state=42  
)  
Important  
n_estimators
```

learning_rate

max_depth

10. XGBoost

```
from xgboost import XGBClassifier  
model = XGBClassifier(  
    n_estimators=100,  
    learning_rate=0.1,  
    max_depth=6,  
    subsample=1,  
    colsample_bytree=1,  
    random_state=42  
)  
Important  
n_estimators
```

learning_rate

max_depth

subsample

colsample_bytree

11. LightGBM

```
from lightgbm import LGBMClassifier  
model = LGBMClassifier(  
    n_estimators=100,  
    learning_rate=0.1,  
    num_leaves=31,  
    max_depth=-1  
)  
Important  
num_leaves
```

learning_rate

max_depth

n_estimators

12. CatBoost

```
from catboost import CatBoostClassifier  
model = CatBoostClassifier(  
    iterations=100,  
    learning_rate=0.1,  
    depth=6,  
    verbose=0  
)  
Important  
iterations
```

learning_rate

depth

13. Support Vector Machine

```
from sklearn.svm import SVC  
model = SVC(  
    kernel='rbf',  
    C=1,  
    gamma='scale'  
)  
Important  
kernel
```

C

gamma

14. Naive Bayes

Gaussian

```
from sklearn.naive_bayes import GaussianNB
```

```
model = GaussianNB(  
    var_smoothing=1e-9
```

)

Multinomial

```
from sklearn.naive_bayes import MultinomialNB
```

```
model = MultinomialNB(  
    alpha=1.0  
)
```

Bernoulli

```
from sklearn.naive_bayes import BernoulliNB
```

```
model = BernoulliNB(  
    alpha=1,  
    binarize=0  
)
```

15. KNN

```
from sklearn.neighbors import KNeighborsClassifier  
model = KNeighborsClassifier(  
    n_neighbors=5,  
    metric='minkowski',  
    p=2,  
    weights='uniform'  
)  
Important  
n_neighbors
```

metric

p

weights

16. K-Means

```
from sklearn.cluster import KMeans  
model = KMeans(  
    n_clusters=3,  
    init='k-means++',  
    n_init='auto',  
    max_iter=300,  
    random_state=42  
)  
Important  
n_clusters
```

init

n_init

max_iter

random_state
Useful Attributes
model.labels_

model.cluster_centers_

model.inertia_

17. Hierarchical Clustering

```
from sklearn.cluster import AgglomerativeClustering  
model = AgglomerativeClustering(  
    n_clusters=3,  
    linkage='ward',  
    metric='euclidean'  
)  
Important  
n_clusters
```

linkage

metric

distance_threshold

Dendrogram

```
from scipy.cluster.hierarchy import linkage  
from scipy.cluster.hierarchy import dendrogram  
Z = linkage(X, method='ward')
```

dendrogram(Z)

18. DBSCAN

```
from sklearn.cluster import DBSCAN  
model = DBSCAN(  
    eps=0.5,  
    min_samples=5,  
    metric='euclidean'  
)  
Important  
eps
```

min_samples

metric
Useful Attribute
model.labels_

```
Standard Scaling (Used Before Many Algorithms)
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
```

```
Train-Test Split
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
Common Model Pipeline
# Import

# Load Dataset

# Train Test Split

# Feature Scaling (if needed)

# Create Model

# Train

model.fit(X_train, y_train)

# Predict

y_pred = model.predict(X_test)

# Evaluate
```

What is a Pipeline?
Definition
A Pipeline is a Scikit-learn object that combines multiple preprocessing steps and the final model into one sequential workflow.
Instead of calling every step manually,
->Raw Data
Step 1
(Imputer)
Step 2
(Encoder)
Step 3
(StandardScaler)
Step 4
(PCA)
Step 5
(Logistic Regression)
->Prediction

from sklearn.pipeline import Pipeline

```
pipe = Pipeline([
    ("step_name", transformer),
    ("step_name", transformer),
    ("model", estimator)
])
Eg: pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression())
])
```

Pipeline Methods		
Function	Purpose	Used During
fit(X,y)	Learns preprocessing parameters and trains model	Training
predict(X)	Predicts labels after preprocessing	Testing
predict_proba(X)	Returns class probabilities	Classification
transform(X)	Applies only preprocessing (if last step is transformer)	Preprocessing
fit_transform(X)	Fit + Transform	Training
score(X,y)	Returns model score	Evaluation
set_params()	Change hyperparameters	Grid Search
get_params()	View all parameters	Debugging
named_steps	Access individual steps	Inspection

Eg: pipe.fit(X_train,y_train)

ColumnTransformer:

from sklearn.compose import ColumnTransformer

```
ColumnTransformer([
    (
        "name",
        transformer,
        columns
    )
])
Eg : preprocessor = ColumnTransformer([
    (
        "num",
        StandardScaler(),
        ["Age", "Salary"]
    ),
    (
        "cat",
        OneHotEncoder(),
        ["Gender", "City"]
    )
])
```

ColumnTransformer Methods

Function	Purpose
fit()	Learn preprocessing parameters
transform()	Apply transformations
fit_transform()	Fit + Transform
get_feature_names_out()	Return transformed feature names
get_params()	View parameters
set_params()	Modify parameters

Complete Production Pipeline:

```
numeric = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric, numerical_columns),
    ("cat", categorical, categorical_columns)
])

model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier())
])

model.fit(X_train, y_train)

pred = model.predict(X_test)
```

Algorithms That Require Feature Scaling

Algorithm	Scaling Required?
Linear Regression	⚠️ Recommended (especially with regularization)
Ridge	✅ Yes
Lasso	✅ Yes
ElasticNet	✅ Yes
Logistic Regression	✅ Yes
KNN	✅ Yes
SVM	✅ Yes
K-Means	✅ Yes
Hierarchical Clustering	✅ Yes
DBSCAN	✅ Yes
Decision Tree	❌ No
Random Forest	❌ No
AdaBoost	❌ No
Gradient Boosting	❌ No
XGBoost	❌ No
LightGBM	❌ No
CatBoost	❌ No
Naive Bayes	⚠️ Depends on the variant and data

Machine Learning Preprocessing Pipeline (Complete Interview Notes)

Raw Data

- ↓
1. Exploratory Data Analysis (EDA)

2. Data Cleaning

3. Handle Missing Values

4. Remove Duplicates

5. Handle Outliers

6. Train-Test Split

7. Encode Categorical Features

8. Feature Scaling

9. Feature Engineering

10. Feature Selection

11. Handle Class Imbalance

12. Cross Validation

13. Model Training

14. Hyperparameter Tuning

15. Model Evaluation

16. Model Saving & Deployment

1. Exploratory Data Analysis (EDA)

Purpose

Understand the dataset before making any modifications.

What to check

- Dataset size (shape)
- Feature names
- Data types
- Missing values
- Duplicate records
- Target distribution
- Statistical summary
- Correlation between features
- Data distribution

Important Functions

df.shape
df.head()
df.info()
df.describe()
df.columns
df.dtypes
df.isnull().sum()
df.duplicated().sum()

2. Data Cleaning

Purpose

Remove incorrect, inconsistent, or unnecessary data.

Tasks

- Remove irrelevant columns
- Fix incorrect datatypes
- Standardize text values
- Rename columns
- Remove invalid records
- Correct formatting inconsistencies

Functions

drop()
astype()
replace()
rename()

3. Handle Missing Values

Purpose

Replace or remove missing values without introducing bias.

Numerical Features

- Mean (normally distributed data)
- Median (skewed data or outliers)
- KNN Imputer
- Iterative Imputer

Categorical Features

- Mode
- Constant value ("Unknown")

Remove Missing Data

dropna()
Scikit-learn
SimpleImputer()
KNNImputer()
IterativeImputer()

4. Remove Duplicates

Purpose

Duplicate records can bias the model and distort statistics.

Functions

df.duplicated()
df.drop_duplicates()

5. Handle Outliers

Purpose

Identify and treat unusually large or small values that can negatively affect model performance.

Detection Methods

- IQR Method
- Z-Score
- Box Plot
- Percentile Method

Treatment

- Remove
- Capping (Winsorization)
- Log Transformation
- Robust Scaling

Formula

$$IQR = Q3 - Q1$$

Outlier if

$$x < Q1 - 1.5(IQR)$$

or

$$x > Q3 + 1.5(IQR)$$

6. Train-Test Split

Purpose

Separate unseen data for unbiased evaluation.

Common Split

- 80 : 20
- 70 : 30

Function

train_test_split(
X,
Y,
test_size=0.2,
random_state=42
)

Important: Split the data **before fitting** any scaler, encoder, or imputer to prevent **data leakage**.

7. Encode Categorical Features

Purpose

Machine Learning models require numerical input, so categorical variables must be converted into numbers.

Methods

Label Encoding

Assigns integer values.

LabelEncoder()

Use for:

- Ordinal features
- Target labels

One-Hot Encoding

Creates one binary column per category.

OneHotEncoder()

pd.get_dummies()

Use for:

- Nominal features

Ordinal Encoding

Maps ordered categories while preserving order.

OrdinalEncoder()

8. Feature Scaling

Purpose

Bring all features to a similar scale so that no feature dominates due to its magnitude.

Standardization

Centers data around zero with unit variance.

$$z = \frac{x - \mu}{\sigma}$$

StandardScaler()

Normalization

Scales data between 0 and 1.

$$x' = \frac{x - \min}{\max - \min}$$

MinMaxScaler()

Robust Scaling

Uses median and IQR.

Less sensitive to outliers.

RobustScaler()

Algorithms requiring scaling

- Logistic Regression
- KNN
- SVM

- K-Means
- PCA
- Neural Networks
- DBSCAN
- Hierarchical Clustering

No Scaling Required

- Decision Tree
- Random Forest
- XGBoost
- LightGBM
- CatBoost

9. Feature Engineering

Purpose

Create new informative features from existing data to improve model performance.

Common Techniques

- Date & Time decomposition
- Interaction features
- Polynomial features
- Aggregation features
- Text feature extraction
- Domain-specific transformations

Tool

PolynomialFeatures()

10. Feature Selection

Purpose

Remove irrelevant or redundant features.

Benefits

- Faster training
- Less overfitting
- Better interpretability

Filter Methods

- Correlation
- Chi-Square
- ANOVA
- Mutual Information

Wrapper Methods

- Recursive Feature Elimination (RFE)
- Sequential Feature Selection

Embedded Methods

- Lasso
- Tree Feature Importance

11. Handle Class Imbalance

Purpose

Prevent the model from being biased toward the majority class.

Techniques

Oversampling

SMOTE()

RandomOverSampler()

Undersampling

RandomUnderSampler()

Class Weights

class_weight="balanced"

Always apply only on the training data.

12. Cross Validation

Purpose

Evaluate model stability by training and validating on multiple data splits.

Common Types

- K-Fold
- Flask
- FastAPI
- Streamlit
- Docker
- AWS
- Azure
- Google Cloud

★ **Important Preprocessing Imports**

```
from sklearn.model_selection import (
    train_test_split,
    GridSearchCV,
    RandomizedSearchCV,
```

```
cross_val_score,
KFold,
StratifiedKFold
)
```

```
from sklearn.preprocessing import (
    StandardScaler,
    MinMaxScaler,
    RobustScaler,
    LabelEncoder,
    OneHotEncoder,
    OrdinalEncoder,
    PolynomialFeatures
)
```

```
from sklearn.impute import (
    SimpleImputer,
```

- Stratified K-Fold

Functions

cross_val_score()

KFold()

StratifiedKFold()

Usually

- 5 Fold
- 10 Fold

13. Model Training

Purpose

Train the algorithm using training data.

model.fit(X_train, y_train)

14. Hyperparameter Tuning

Purpose

Find the best combination of model hyperparameters.

Methods

Grid Search

GridSearchCV()

Random Search

RandomizedSearchCV()

Bayesian Optimization

- Optuna
- Hyperopt

15. Model Evaluation

Purpose

Measure model performance using appropriate metrics.

Classification

- Accuracy
- Precision
- Recall
- F1 Score
- ROC-AUC
- Confusion Matrix

Regression

- MAE
- MSE
- RMSE
- R² Score

Clustering

- Silhouette Score
- WCSS (Inertia)
- Davies-Bouldin Index
- Calinski-Harabasz Index

16. Model Saving & Deployment

Purpose

Save the trained model for future inference and deployment.

Save Model

joblib.dump()

pickle.dump()

Load Model

joblib.load()

pickle.load()

Deployment Frameworks

```
KNNImputer,
IterativeImputer
)
```

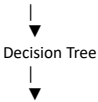
```
from sklearn.feature_selection import (
    SelectKBest,
    RFE,
    chi2,
    f_classif,
    mutual_info_classif
)
```

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
```

Next Chapter : Hyperparameter Tuning

Step 1: We Built Our First ML Model

Suppose you have a dataset.
Customer Data



Accuracy = 82%
You think,
"Can I improve this accuracy?"
Probably yes.

Step 2: Why Isn't Every Decision Tree the Same?

Look at these two trees.

Tree 1
Max Depth = 2
Root
/ \
Yes No
Very small tree.
Accuracy
78%

Tree 2
Max Depth = 15
Huge Tree
Lots of Splits
Accuracy
95% (Training)

70% (Testing)
It memorized the training data.

This is **Overfitting**.

Now another tree.
Max Depth = 6
Training
91%
Testing
89%
Better.

Question:
Who told us that Max Depth = 6 is the best?
Nobody.
We guessed.
That is the problem.

Step 3: Parameters vs Hyperparameters
Before understanding tuning, understand these two terms.

Parameters

These are **learned automatically by the model**.
Example
Linear Regression

$$y = mx + c$$

The model learns

- m, c

These are **parameters**.
You never set them.

Decision Tree
The model automatically learns

- split locations
- leaf values

These are parameters.

Hyperparameters
These are **decisions made before training**.

Example
DecisionTreeClassifier(
 max_depth=5,
 criterion='gini'
)
You selected
max_depth = 5
criterion = gini
The model never changes these.
These are hyperparameters.

Step 4: Why Hyperparameter Tuning?

Suppose you try
max_depth = 2
Accuracy
76%

Try
max_depth = 4
Accuracy
83%

Try
max_depth = 8
Accuracy
91%

Try
max_depth = 20
Accuracy
74%
Now accuracy decreased.
Different hyperparameters
↓
Different models
↓
Different accuracy.
So we need a method that automatically finds
the best combination.

Step 5: First Idea — Manual Search

Initially people simply did
for depth in [2,4,6,8]:
Train
Check Accuracy
Remember Best
Repeat.
Very slow.

Someone thought,
"Why not automate this?"
This became
Grid Search

GridSearchCV

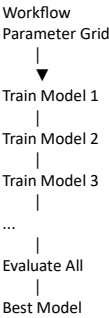
Definition
Grid Search tries **every possible combination** of hyperparameters.

Suppose
max_depth = [2,4,6]

criterion = ['gini','entropy']
Possible combinations

Depth	Criterion
2	gini
2	entropy
4	gini
4	entropy
6	gini
6	entropy

Grid Search trains
ALL
6 models.



Code
from sklearn.model_selection import GridSearchCV

```
params = {  
    "max_depth":[2,4,6],  
    "criterion":["gini","entropy"]  
}
```

```
grid = GridSearchCV(  
    estimator=DecisionTreeClassifier(),  
    param_grid=params,  
    cv=5,  
    scoring="accuracy"  
)
```

grid.fit(X_train,y_train)

Question
Why
cv=5
?
Because
Grid Search does **Cross Validation**.
Which brings us to another important topic.

Step 6: Cross Validation
Suppose

1000 samples.
Normally
800 Train

200 Test
Accuracy
91%
Looks good.
But what if those 200 test samples were unusually easy?
Maybe your model isn't actually 91%.
Maybe it's 85%.
One train-test split can be misleading.

Solution
Instead of one split,
train multiple times.

5-Fold Cross Validation

Suppose
1000 samples.
Divide into
5 equal parts.
Fold 1

Fold 2

Fold 3

Fold 4

Fold 5
Now
Iteration 1
Test = Fold 1

Train = Remaining

Iteration 2
Test = Fold 2

Train = Remaining
Repeat.
Fold1

Fold2

Fold3

Fold4

Fold5
Each fold becomes the test set exactly once.

Suppose accuracies are
89

91

90

92

88
Average
90%

This is much more reliable than a single train-test split.

Why use Cross Validation?

- More stable estimate
- Less dependent on one random split
- Better use of data
- Reduces variance in performance estimates

Now return to Grid Search.
Grid Search performs
Cross Validation
for every hyperparameter combination.
Imagine
6 combinations
×
5 folds
=
30 model trainings.
Already expensive.

Now imagine
20 hyperparameters.
Each with
10 values.
Grid Search becomes impossible.

This led to the next idea.
RandomizedSearchCV

Instead of checking
every combination,

randomly pick
some combinations.
Suppose
Total possible combinations
5000
Grid Search
↓
Train
5000 models.

Random Search
↓
Randomly pick
100 combinations.
Much faster.

Workflow
Parameter Space
↓
Randomly Select
↓
Train
↓
Evaluate
↓
Best Found

Advantages

- Much faster
- Works well on large search spaces
- Often finds nearly the same performance as Grid Search

Disadvantage

May miss the absolute best combination.

Code

```
RandomizedSearchCV(  
    estimator=model,  
    param_distributions=params,  
    n_iter=100,  
    cv=5  
)
```

Notice
n_iter
means
How many random combinations to try.

Step 7: Can We Be Even Smarter?

Researchers noticed something interesting.

Suppose
First 20 experiments
all show
learning_rate = 0.1
works well.
Why keep testing
learning_rate = 5
Clearly bad.
Instead,
learn from previous experiments.
This idea gave birth to
Bayesian Optimization.

Optuna

Definition

Optuna is a modern hyperparameter optimization library that uses **Bayesian Optimization** to intelligently search the hyperparameter space.
Instead of random guessing, it learns from previous trials and focuses on promising regions.

Workflow

Try One Combination
↓
Evaluate
↓
Learn
↓
Suggest Better Combination
↓
Evaluate Again
↓
Learn Again
↓
Repeat
Unlike Random Search,
Optuna becomes smarter after every experiment.

Code

```
import optuna
```

```
def objective(trial):
```

```
    max_depth = trial.suggest_int("max_depth",2,10)
```

```
    learning_rate = trial.suggest_float(  
        "learning_rate",  
        0.001,  
        0.3  
    )
```

...
Advantages
• Very fast • Intelligent search • State-of-the-art • Widely used in industry and Kaggle

Hyperopt Hyperopt is another Bayesian Optimization library. Very similar to Optuna. Instead of <code>trial.suggest_int()</code> it uses <code>hp.choice()</code> <code>hp.uniform()</code> <code>hp.loguniform()</code> Conceptually, both solve the same problem. Nowadays, Optuna is generally more popular because of its simpler API and visualization tools.

Evolution of Hyperparameter Tuning Manual Trial & Error ↓ Grid Search ↓ Random Search ↓ Bayesian Optimization ├── Optuna └── Hyperopt Each method was created to solve the limitations of the previous one.

Bias-Variance Tradeoff (The Most Important Concept) Imagine you're preparing for an exam.
Student A Studies only one day. Gets every difficult question wrong. This student hasn't learned enough. This is High Bias.
Student B Memorizes the entire textbook. Even remembers spelling mistakes. Scores 100% on practice papers. Fails in the real exam. This is High Variance.

Student C Studies concepts. Practices different questions. Understands patterns. Performs well everywhere. This is the balance we want.
What is Bias? Definition Bias is the error caused because the model is too simple to learn the true relationship in the data. Example Decision Tree <code>max_depth = 1</code> Very simple. Cannot capture complex patterns. Result • Low Training Accuracy • Low Testing Accuracy This is called Underfitting.

What is Variance? Definition Variance is the error caused because the model becomes too complex and memorizes the training data. Example <code>max_depth = 100</code> Huge tree. Training Accuracy 100% Testing Accuracy 70% This is Overfitting.
--

Goal Find the balance. Model Complexity ↓
--

Too Simple → High Bias ↓ Optimal Model ↓ Too Complex → High Variance Hyperparameter tuning is essentially trying to find this sweet spot.
Complete Pipeline Choose Model ↓ Select Hyperparameters ↓ Cross Validation ↓ Evaluate Performance ↓ Tune Hyperparameters ↓ Best Model ↓ Final Test Evaluation

Interview Questions 1. Why don't we evaluate hyperparameters on the test set? Because the test set must remain unseen until the very end. We tune using the training set (often with cross-validation). Using the test set for tuning causes data leakage and gives an overly optimistic estimate of performance.
2. Why does GridSearchCV use Cross Validation? Because a single train-validation split may be misleading. Cross-validation provides a more reliable estimate of how each hyperparameter combination will generalize.
3. Why is Random Search often preferred over Grid Search? Grid Search evaluates every combination, even unpromising ones. Random Search explores the space more efficiently and often achieves similar performance with far fewer evaluations.
4. Why are Optuna and Hyperopt becoming popular? Because they use Bayesian Optimization, which learns from previous trials and intelligently searches promising regions instead of blindly trying combinations.
5. What is the Bias-Variance Tradeoff? It is the balance between underfitting (high bias) and overfitting (high variance). The goal of hyperparameter tuning is to find model settings that minimize generalization error by achieving the right balance.

Next Chapter :Feature Selection

Step 1: You Built Your First Dataset

Suppose you are building a model to predict whether a customer will buy a product. Your dataset has

Feature	Description
Age	Customer Age
Salary	Annual Income
Gender	Male/Female
City	Customer City
Mobile Number	Customer Phone
Email	Email Address
Name	Customer Name
Purchase Amount	Previous Purchase
Last Login	Last Login Date
Purchased	Target

Looks good.

You train a Random Forest.

Accuracy

84%

Step 2: More Features ≠ Better Model Now suppose you add Customer ID PAN Number Passport Number Blood Group Favorite Color Vehicle Number Now your dataset has 100 features You expect More Information
--

↓

Better Model
But instead
Accuracy drops.
Training becomes slower.
Why?
Because many features contain **no useful information**.

Imagine asking a doctor:
"Will this patient have heart disease?"
Instead of giving

- Blood Pressure
- Cholesterol
- Age

you also provide

- Favorite Movie
- Shoe Size
- Aadhaar Number

These extra features don't help.
Sometimes they even confuse the model.

This gives rise to a very important question:
Which features should we keep?
That is exactly what **Feature Selection** solves.

What is Feature Selection?

Definition

Feature Selection is the process of selecting the most useful features from a dataset while removing irrelevant, redundant, or noisy features.

Notice something important.

We are **not creating new features**.

We are simply deciding

Keep

or

Remove

Suppose
Original Dataset

Features

Age
Salary
Gender
City
Phone
Email
Purchase

After Feature Selection

Features

Age
Salary
Purchase

The useless columns are removed.

Why Do We Need Feature Selection?

Benefits

- ✓ Faster training
- ✓ Lower memory usage
- ✓ Reduces overfitting
- ✓ Improves interpretability
- ✓ Sometimes improves accuracy

The Big Question

How do we know
which feature is important?
Different researchers proposed different ideas.
These ideas became
Filter

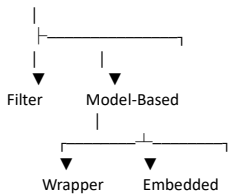
Wrapper

Embedded

Each solves the same problem differently.

Feature Selection Hierarchy

Feature Selection



Let's understand why each one exists.

Method 1 : Filter Methods

The Story

Suppose you have
100 features.
Training a model for every feature combination would take hours.
Someone asked:
"Can we judge a feature **without training any model**?"
Answer:
Yes.
Use **statistics**.
This became the **Filter Method**.

Definition

Filter methods evaluate every feature using **statistical tests**.

No machine learning model is trained.

Think of it like a **preliminary screening test**.

Workflow

Dataset



Statistical Test



Assign Score



Keep Best Features

Fast.

Simple.

No model involved.

There are three major statistical tests.

1. Chi-Square Test

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$

$$\chi^2(3) = 5.04, p = 0.169$$

The sample does not provide enough evidence that the four proportions differ

Observed A

Observed A

Observed B

Observed B

Observed C

Observed C

Observed D

Observed D

AContribution **1.96**

Observed 18Expected 25

BContribution **0.36**

Observed 22Expected 25

CContribution **0.16**

Observed 27Expected 25

DContribution **2.56**

Observed 33Expected 25

Total contribution 5.04Below 5% cutoff

The Story

Suppose you want to predict
Purchased

Yes / No

One feature is

Gender

Male

Female

Question:

Does Gender affect Purchase?

If yes,

keep it.

If no,

remove it.

How do we measure that?

Chi-Square.

Definition

Chi-Square measures **how strongly a categorical feature is related to a categorical target**.

If the relationship is strong,

High Score.

If unrelated,

Low Score.

Example

Gender Purchased

Male Yes

Male Yes

Female No

Male Yes

Clearly related.

High Chi-Square score.

Suppose

Blood Group Purchased

A Yes

Blood Group Purchased

B No
AB Yes
O No

No clear pattern.
Low score.
Remove.

Use When

- ✔ Classification
- ✔ Categorical Features

Code

```
from sklearn.feature_selection import SelectKBest, chi2
```

```
selector = SelectKBest(  
    score_func=chi2,  
    k=10  
)
```

```
X_new = selector.fit_transform(X,y)
```

2. ANOVA

$$F = \frac{\text{between-group variation}}{\text{within-group variation}}$$

Evidence of difference

Group mean spacing

Group mean spacing

Within-group spread

Within-group spread

Suppose

Feature

Salary

Target

Purchased

Yes

No

Question

Do buyers and non-buyers have significantly different salaries?

If yes,

Salary is important.

If no,

remove it.

ANOVA answers

"Do different classes have significantly different means?"

If

Average Salary

Purchased

₹10 Lakh

Not Purchased

₹2 Lakh

Huge difference.

Important Feature.

If

Both averages

₹5 Lakh

Feature isn't helping.

Use When

- ✔ Numerical Feature
- ✔ Classification

Code

```
SelectKBest(  
    score_func=f_classif,  
    k=10  
)
```

3. Mutual Information

The previous two methods have limitations.

Suppose

Relationship looks like

o

o o

o o

Not linear.

ANOVA may fail.

Chi-Square may fail.

Need something smarter.

Mutual Information asks

"How much information does this feature give me about the target?"

The more information,
the higher the score.

Example

Study Hours

↓

Exam Marks

Very informative.

High score.

Example

Shoe Size

↓

Exam Marks

No information.

Low score.

Best Part

Works for

- Linear relationships
- Nonlinear relationships
- Numerical
- Categorical

Code

```
SelectKBest(  
    score_func=mutual_info_classif,  
    k=10  
)
```

Filter Methods Summary

Filter methods never train a model.

They only ask

"Does this feature look statistically useful?"

Fast.

Cheap.

Great for very large datasets.

Wrapper Methods

Researchers noticed something.

A feature may individually look weak,

but together with another feature,

it may become extremely useful.

Statistics cannot detect this.

Need a real model.

Thus,

Wrapper Methods were created.

Definition

Wrapper methods train a machine learning model repeatedly using different subsets of

features.

The subset giving the best performance is selected.

Workflow

Dataset

|

Select Feature Subset

|

Train Model

|

Measure Accuracy

|

Repeat

|

Choose Best Features

Much slower.

But usually more accurate.

Recursive Feature Elimination (RFE)

This is the most popular Wrapper Method.

Suppose

Features

Age

Salary

Gender

City

Purchase History

Train a model.

Model importance

Age → 90

Salary → 80

Gender → 20

City → 5

Purchase → 70

Least useful

City

Remove it.
Train again.
Again remove the weakest feature.
Repeat until only the required number of features remain.

Workflow
Train Model
|
Rank Features
|
Remove Worst Feature
|
Train Again
|
Repeat

Code
from sklearn.feature_selection import RFE

```
selector = RFE(  
    estimator=LogisticRegression(),  
    n_features_to_select=5  
)
```

X_new = selector.fit_transform(X,y)

Why is RFE slow?
Because if you have
100 features,
it may train the model dozens of times,
removing one feature (or a small set) in each iteration until the target number of features remains.

Embedded Methods

Researchers then asked
"Why train one model for prediction and another process for feature selection?"
Why not do both together?
This created
Embedded Methods.

Definition
Feature selection happens **during model training itself**.
No separate step.

Workflow
Train Model
|
Model Learns
|
Ranks Features
|
Removes Weak Ones

Much faster than Wrapper.
Usually more accurate than Filter.

Lasso Regression

Remember
L1 Regularization.
Loss Function
Loss

+

$\lambda \sum |w|$
The penalty forces
some coefficients to become
exactly
0
Feature
whose coefficient becomes zero
↓
No contribution.
↓
Automatically removed.

Example
Coefficients
Age = 2.5

Salary = 1.2

City = 0

Gender = 0

Purchase = 3
Selected
Age

Salary

Purchase

Lasso performs
Training

•

Feature Selection
at the same time.

Tree Feature Importance

Decision Trees constantly ask
Which feature gives me the best split?
The more useful a feature is,
the more often it is chosen for important splits,
and the more it reduces impurity.
Examples:

- Decision Tree
- Random Forest
- XGBoost
- LightGBM
- CatBoost

All of these can compute feature importance automatically.

Code
model = RandomForestClassifier()

model.fit(X,y)

model.feature_importances_

Output
Age 0.42

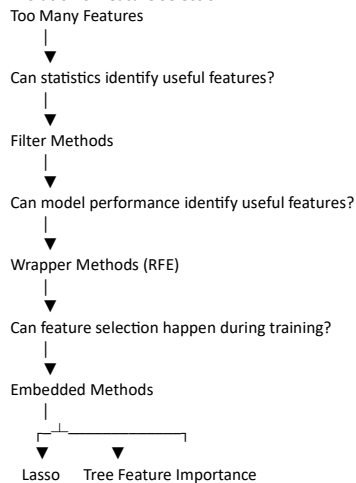
Salary 0.30

City 0.05

Gender 0.08

Purchase 0.15
Higher score
↓
More important feature.

Evolution of Feature Selection



Comparison

Method	Idea	Uses Model?	Speed	Accuracy
Filter	Use statistical tests to score each feature independently	✗ No	★★★★★	★★★★★
Wrapper (RFE)	Try different feature subsets by repeatedly training a model	✓ Yes	★★	★★★★★
Embedded	Let the model perform feature selection while training	✓ Yes	★★★★	★★★★★

Interview Summary (30-Second Answer)

Feature Selection is the process of selecting the most relevant features while removing irrelevant or redundant ones to improve model performance, reduce overfitting, and decrease training time. There are three main approaches: Filter methods, which use statistical tests like Chi-Square, ANOVA, and Mutual Information without training a model; Wrapper methods, such as Recursive Feature Elimination (RFE), which repeatedly train a model on different feature subsets; and Embedded methods, where feature selection occurs during model training itself, such as Lasso Regression using L1 regularization and Tree-based Feature Importance in Decision Trees, Random Forest, and XGBoost.

NumPy Interview Cheat Sheet

NumPy (Numerical Python) is the fundamental library for numerical computing in Python. It provides the **ndarray (N-dimensional array)**, which is faster and more memory-efficient than Python lists due to contiguous memory storage and vectorized operations.

1. Import

import numpy as np

2. Why NumPy?

- Faster than Python lists
- Less memory consumption
- Supports vectorized operations
- Broadcasting support
- Powerful mathematical functions
- Foundation of Pandas, Scikit-learn, TensorFlow, PyTorch

3. Creating Arrays

np.array([1,2,3])

np.array([[1,2],[3,4]])
np.zeros((3,4))
np.ones((2,3))
np.empty((2,2))
np.eye(4)
np.identity(4)
np.arange(1,10,2)
np.linspace(0,1,5)
np.random.rand(3,3)
np.random.randn(3,3)
np.random.randint(1,100,(3,4))
np.full((3,3),7)

4. Important Array Attributes

arr.shape # Dimensions
arr.ndim # Number of dimensions
arr.size # Total elements
arr.dtype # Data type
arr.itemsize # Bytes per element
arr.nbytes # Total memory
arr.T # Transpose

5. Changing Shape

arr.reshape(2,3)

arr.flatten()
arr.ravel()
arr.resize((3,2))
arr.transpose()

Interview Difference

flatten()	ravel()
Returns copy	Returns view (if possible)
More memory	More efficient

6. Indexing & Slicing

arr[2]

arr[-1]

arr[1:5]

arr[:]

arr[:,1]

arr[1,:]

arr[1:3,2:4]
Boolean Indexing
arr[arr>5]
Fancy Indexing
arr[[1,3,5]]

7. Array Operations

a+b
a-b
a*b
a/b
a**2
np.sqrt(a)
np.power(a,2)
np.exp(a)
np.log(a)
np.abs(a)
All operations are **element-wise**.

8. Aggregation Functions

np.sum(arr)
np.mean(arr)
np.median(arr)
np.std(arr)

np.var(arr)
np.min(arr)
np.max(arr)
np.argmax(arr)
np.argmin(arr)
np.prod(arr)
np.percentile(arr,90)
np.quantile(arr,0.5)
Along Axis
np.sum(arr,axis=0)
np.sum(arr,axis=1)

9. Statistical Functions

np.mean()
np.median()
np.std()
np.var()
np.corrcoef()
np.cov()
np.percentile()
np.quantile()

10. Mathematical Functions

np.sqrt()
np.square()
np.power()
np.exp()
np.log()
np.log10()
np.sin()
np.cos()
np.tan()
np.ceil()
np.floor()
np.round()

11. Random Module

np.random.seed(42)

np.random.rand()
np.random.randn()
np.random.randint()
np.random.choice()
np.random.shuffle()
np.random.permutation()
np.random.normal()

np.random.uniform()

Interview

Difference
rand()
Uniform Distribution
randn()
Normal Distribution

12. Sorting & Searching

np.sort(arr)
np.argsort(arr)
np.argmax(arr)
np.argmin(arr)
np.where(arr>5)
np.unique(arr)
np.count_nonzero(arr)

13. Combining Arrays

np.concatenate()

np.vstack()

np.hstack()

np.dstack()

np.column_stack()

np.row_stack()

14. Splitting Arrays

np.split()
np.hsplit()
np.vsplit()
np.array_split()

15. Broadcasting (Very Important)

Definition

Broadcasting allows NumPy to perform operations on arrays of different shapes **without explicitly copying data**, as long as the shapes are compatible.

Example
a = np.array([[1],[2],[3]])

b = np.array([10,20,30])

a + b
Output
[[11 21 31]
[12 22 32]
[13 23 33]]

Broadcasting Rules

1. Compare dimensions from the **rightmost** side.
2. Dimensions are compatible if:
 - They are equal, or
 - One of them is 1.
3. Otherwise, broadcasting fails.

16. Matrix Operations

np.dot(A,B)

A @ B

np.matmul(A,B)

np.linalg.inv(A)

np.linalg.det(A)

np.linalg.eig(A)

np.trace(A)

17. Linear Algebra

np.linalg.solve()
np.linalg.norm()
np.linalg.matrix_rank()
np.linalg.svd()
np.linalg.qr()

18. Useful Utility Functions

np.clip()
np.repeat()
np.tile()
np.insert()
np.delete()
np.append()
np.roll()
np.flip()
np.diag()

19. Handling Missing Values

np.nan()
np.isnan()
np.nanmean()
np.nanmedian()
np.nanstd()
np.nan_to_num()

20. Boolean Functions

np.any()
np.all()
np.logical_and()
np.logical_or()
np.logical_not()

21. Copy vs View (Very Important)

view = arr.view()

copy = arr.copy()

View

- Shares memory
- Changes reflect in original array

Copy

- Independent memory
- No effect on original array

22. Axis Concept (Most Asked)

For a 2D array:

[[1 2 3]

[4 5 6]]

axis=0

Column-wise operation

↓

Result

[5 7 9]

axis=1

Row-wise operation

↓

Result

[6 15]

23. Data Types

np.int32
np.int64
np.float32
np.float64
np.bool
np.str_
np.object_
Convert
arr.astype(np.float32)

24. Memory Optimization

Specify datatype
np.array([1,2,3],dtype=np.int8)

Uses less memory than
int64

25. Time Complexity

Operation	Complexity
Indexing	O(1)
Slicing	O(k)
Sum	O(n)
Mean	O(n)
Max	O(n)
Sort	O(n log n)
Matrix Multiplication	O(n³) (standard algorithm)

★ Most Important Interview Functions

Array Creation

array()
zeros()
ones()
eye()
identity()
arange()
linspace()
full()

Shape Manipulation

reshape()
flatten()
ravel()
transpose()

Statistics

sum()
mean()
median()
std()
var()
min()
max()
argmin()
argmax()

Math

sqrt()
power()
log()
exp()
abs()

Random

seed()
rand()
randn()
randint()
choice()
shuffle()
normal()

Searching

where()
unique()
sort()
argsort()
count_nonzero()

Matrix

dot()
matmul()
inv()
det()
eig()

★ Most Asked Interview Questions

1. Why is NumPy faster than Python lists?

- Stores homogeneous data types.
- Uses contiguous memory allocation.
- Vectorized operations implemented in optimized C.
- Reduces Python-level loops.

2. Difference between reshape(), resize(), flatten(), and ravel()?

Function	Changes Original	Copy/View
reshape()	❌ No	Usually View
resize()	✅ Yes	Modifies Original
flatten()	❌ No	Copy
ravel()	❌ No	View (if possible)

3. Difference between np.array() and Python List?

Python List	NumPy Array
Slower	Faster

Python List

More memory
Mixed data types
No vectorization
Limited mathematical operations

NumPy Array

Less memory
Homogeneous data type
Vectorized operations
Rich mathematical operations

4. What is Vectorization?

Performing operations on an entire array **without explicit Python loops**, using optimized NumPy implementations.

5. What is Broadcasting?

A mechanism that enables arithmetic operations between arrays of different but compatible shapes without explicit replication of data.

6. Difference between rand() and randn()?

rand()	randn()
Uniform distribution	Normal (Gaussian) distribution
Values in [0, 1)	Mean = 0, Std = 1

7. Difference between View and Copy?

View	Copy
Shares memory	Separate memory
Faster	Slightly slower
Changes affect original	Independent

8. Difference between np.concatenate(), vstack(), and hstack()?

- concatenate() → Join arrays along a specified axis.
- vstack() → Stack arrays vertically (row-wise).
- hstack() → Stack arrays horizontally (column-wise for 2D arrays).

★ Things to Remember for ML Interviews

- Always use **vectorized operations** instead of Python loops when possible.
- Understand **broadcasting rules** thoroughly.
- Know the difference between **view** and **copy** to avoid unintended modifications.
- Be comfortable with **axis=0** (column-wise) and **axis=1** (row-wise) operations.
- Use reshape(-1, 1) for a **single feature (column vector)** and reshape(1, -1) for a **single sample (row vector)**.
- flatten() returns a **copy**; ravel() returns a **view whenever possible**.
- Use np.random.seed() to make experiments **reproducible**.
- Prefer NumPy operations over Python loops for **better performance and readability**.
- NumPy arrays require **homogeneous data types**, unlike Python lists.
- Many ML libraries (Pandas, Scikit-learn, TensorFlow, PyTorch) internally rely on NumPy arrays, making NumPy a foundational skill for interviews.

Pandas Interview Cheat Sheet

Pandas is the most widely used Python library for **data manipulation, cleaning, analysis, and preprocessing**. It is built on top of **NumPy** and provides two main data structures:

- **Series** (1D labeled array)
- **DataFrame** (2D tabular data)

1. Import

import pandas as pd

2. Why Pandas?

- Easy data manipulation
- Read/write multiple file formats
- Handle missing values
- Data cleaning
- Filtering & grouping
- Time series support
- Built on NumPy

3. Data Structures

Series

s = pd.Series([10,20,30])
One-dimensional labeled array.

DataFrame

```
df = pd.DataFrame({
    "Name":["A","B"],
    "Age":[20,21]
})
```

Two-dimensional table with rows and columns.

4. Reading Data

pd.read_csv()

pd.read_excel()

pd.read_json()

pd.read_sql()

pd.read_parquet()

Example
df = pd.read_csv("data.csv")

5. Saving Data

df.to_csv()

df.to_excel()

df.to_json()

df.to_sql()

6. Basic Information (Most Used)

df.head()

df.tail()

df.sample(5)

df.info()

df.describe()

df.shape

df.columns

df.index

df.dtypes

df.size

df.memory_usage()

df.nunique()

df.count()

7. Selecting Data

Single Column

df["Age"]
or
df.Age

Multiple Columns

df[["Age","Salary"]]

Select Rows

df.iloc[0]

df.iloc[1:5]

df.mean()

df.median()

df.min()

df.max()

df.std()

df.var()

df.count()

df.describe()

18. Value Counts
df["Gender"].value_counts()
Useful for checking class imbalance.

19. Unique Values
df.unique()

df.nunique()

20. GroupBy (Most Important)
df.groupby("Department")
Aggregate
df.groupby("Department")["Salary"].mean()

df.groupby("Department").sum()

df.groupby("Department").count()
Multiple Aggregations
df.groupby("Department").agg({
 "Salary":["mean","max"],
 "Age":"min"
})

21. Merge DataFrames
pd.merge()
Types
how="inner"

how="left"

how="right"

how="outer"

22. Join
df.join()
Join by index.

23. Concatenate
pd.concat()
Row-wise
axis=0
Column-wise
axis=1

24. Pivot Table
pd.pivot_table()
Example
pd.pivot_table(
 df,
 values="Salary",
 index="Department",
 aggfunc="mean"
)

25. Crosstab
pd.crosstab()
Used for frequency tables.

26. Apply Functions
df.apply()

df.map()

df.applymap()
Difference

Function	Works On
map()	Series
apply()	Series/DataFrame
applymap()	Every element (DataFrame)

27. Lambda Function
df["Age"] = df["Age"].apply(lambda x:x+1)

28. DateTime Functions
pd.to_datetime()

.dt.year

.dt.month

.dt.day
.dt.weekday
.dt.hour
29. Correlation df.corr() Useful for feature selection.
30. Memory Optimization df.info(memory_usage="deep")
df.astype("category")
31. Important Iteration Functions df.iterrows()
df.itertuples()
df.items() Avoid loops when possible.
32. Important Boolean Functions df.any()
df.all()
df.where()
df.query()
33. Export df.to_csv()
df.to_excel()
df.to_json()
df.to_sql()
34. Commonly Used Functions in ML head() info() describe() shape columns dtype() isnull() fillna() dropna() drop() rename() astype() value_counts() groupby() merge() concat() corr() sample() sort_values()
🌟 Most Important Interview Functions Reading read_csv() read_excel() read_json()
Data Inspection head() tail() info() describe() shape columns dtypes
Missing Values isnull()
fillna()
dropna()
Cleaning drop()
drop_duplicates()
rename()
astype()
Selection loc

iloc

Analysis

groupby()

agg()

value_counts()

corr()

pivot_table()

Combining

merge()

join()

concat()

★ Most Asked Interview Questions

1. Difference between Series and DataFrame?

Series

DataFrame

1D

2D

Single column

Multiple columns

One data type

Different data types allowed

2. Difference between loc and iloc?

loc

iloc

Label-based

Integer-based

End index included

End index excluded

3. Difference between merge(), join(), and concat()?

Function

Purpose

merge()

SQL-style join on common columns

join()

Join using index

concat()

Stack DataFrames row-wise or column-wise

4. Difference between apply(), map(), and applymap()?

Function	Scope
map()	Series only
apply()	Series or DataFrame
applymap()	Element-wise on entire DataFrame

5. Difference between isnull() and notnull()?

Function	Description
isnull()	Detect missing values
notnull()	Detect non-missing values

6. Difference between value_counts() and unique()?

Function	Description
value_counts()	Frequency of each unique value
unique()	Returns unique values only

7. Difference between groupby() and pivot_table()?

groupby()	pivot_table()
Groups data for aggregation	Creates summarized pivot table
More flexible	Better for reporting

8. Difference between copy() and view()?

- df.copy() creates an independent copy.
- Slicing often returns a **view** (or a copy depending on the operation), which can lead to SettingWithCopyWarning. Use .copy() when you need an independent DataFrame.

- ★ **Things to Remember for ML Interviews**
- head(), info(), describe(), and isnull().sum()** are usually the first functions used after loading a dataset.
 - Prefer **vectorized Pandas operations** over explicit loops (iterrows() is slow).
 - Use **loc** for label-based indexing and **iloc** for integer-based indexing.
 - Use **groupby()** for aggregations and **merge()** for combining datasets.
 - value_counts()** is commonly used to inspect class distribution in classification problems.
 - Use **corr()** to identify highly correlated features before feature selection.
 - Convert appropriate string columns to **category** to reduce memory usage.
 - query()** often makes complex filtering expressions more readable than chained boolean indexing.

Statistics :

Population

A **population** is the complete set of observations or individuals that we are interested in studying.

Sample

A **sample** is a subset of the population selected for analysis.
Instead of studying every customer, we study a representative group.

Population Parameters vs Sample Statistics

Population	Sample
Entire dataset	Portion of dataset
Represented by Greek letters	Represented by Roman letters
Mean = μ	Mean = $\bar{x}$
Variance = σ^2	Variance = s^2
Standard deviation = σ	Standard deviation = s

Interview Question

Q: Why do we divide by (n-1) while calculating sample variance?

Because the sample mean is only an estimate of the population mean. Dividing by (n-1) (known as **Bessel's Correction**) removes the bias and gives an unbiased estimate of the population variance.

3. Types of Data

Understanding data types is important because the choice of statistical methods depends on them.

A. Categorical Data (Qualitative)

Represents categories or labels.
Examples:

- Gender
- Country
- Color
- Product Category
- Department

Machine Learning often requires categorical variables to be encoded into numerical form.

Nominal Data

Categories with **no natural order**.
Examples:

- Blood Group
- City
- Car Brand

You cannot rank them.

Ordinal Data

Categories with a meaningful order.
Examples:

- Low, Medium, High
- Education Level
- Customer Satisfaction (1-5)

Order exists, but the difference between categories is not necessarily equal.

B. Numerical Data (Quantitative)

Represents measurable values.
Examples:

- Age
- Salary
- Height
- Temperature

Discrete Data

Countable values.
Examples:

- Number of Students
- Number of Orders
- Number of Cars

Typically integers.

Continuous Data

Can take any value within a range.
Examples:

- Weight
- Height
- Time

- Distance

Often includes decimals.

Interview Tip

Data Type	Example	Suitable Summary
Nominal	Gender	Mode
Ordinal	Satisfaction Rating	Median
Numerical	Salary	Mean, Median, Std Dev

4. Measures of Central Tendency

Central tendency tells us where the "center" of the data lies.
The three most common measures are:

- Mean
- Median
- Mode

Mean

Definition
The **mean** (average) is the sum of all observations divided by the total number of observations. It represents the central value of the dataset.

Formula

For a population:

$$\mu = \frac{\sum_{i=1}^N x_i}{N}$$

For a sample:

$$\bar{x} = \frac{\sum_{i=1}^n x_i}{n}$$

Where:

- x_i = individual observations
- N = population size
- n = sample size

Properties

- Uses every observation.
- Sensitive to extreme values (outliers).
- Best suited for symmetric distributions.

Advantages

- Easy to compute.
- Uses all available information.
- Widely used in statistical analysis and machine learning.

Disadvantages

- Strongly influenced by outliers.
- Not appropriate for highly skewed distributions.

Interview Questions

When should you use the mean?
Use it when:

- Data is numerical.
- Distribution is approximately normal (symmetric).
- There are few or no extreme outliers.

When should you avoid using the mean?

Avoid it when:

- Data contains many outliers.
- Distribution is highly skewed.
- Data is categorical.

Applications

- Average salary
- Average customer spend
- Average response time
- Average temperature

Median

Definition
The **median** is the middle value after sorting the observations. It divides the dataset into two equal halves.

Formula

If n is odd:

$$Median = x_{\frac{n+1}{2}}$$

If n is even:

$$Median = \frac{x_{\frac{n}{2}} + x_{\frac{n}{2}+1}}{2}$$

Properties

- Not affected by outliers.
- Suitable for skewed data.
- Represents the 50th percentile.

Advantages

- Robust to extreme values.
- Useful for skewed distributions.

Disadvantages

- Does not use all observations.
- Less mathematically convenient than the mean.

Interview Questions

Why use the median instead of the mean?
Because the median is resistant to outliers and provides a better measure of central tendency for skewed data.

Applications

- Income analysis
- House prices
- Medical expenses
- Real estate data

Mode

Definition
The **mode** is the value that appears most frequently in a dataset.

Properties

A dataset can be:

- Unimodal** → One mode
- Bimodal** → Two modes
- Multimodal** → More than two modes
- No mode** → All values occur equally often

Advantages

- Works with categorical data.
- Easy to interpret.

Disadvantages

- May not exist.
- Multiple modes can make interpretation difficult.

Applications

- Most purchased product
- Most common defect type
- Most popular payment method
- Most common customer category

Mean vs Median vs Mode

Feature	Mean	Median	Mode
Uses all observations	✓	✗	✗
Affected by outliers	✓	✗	✗
Suitable for categorical data	✗	✗	✓
Suitable for skewed data	✗	✓	✓ (for frequency)
Mathematical operations	Excellent	Limited	Limited

Choosing the Right Measure

Scenario	Best Choice
Normally distributed numerical data	Mean
Data with outliers	Median
Most frequent category	Mode
Customer income	Median
Exam scores (normal distribution)	Mean
Product category	Mode

Common Interview Questions

1. Why is the mean affected by outliers?
Because every observation contributes equally to the calculation. Extremely large or small values pull the average toward themselves.

2. Which measure should you use for salary analysis?
Median , because salary distributions are typically right-skewed with a small number of very high earners.
3. Can a dataset have multiple modes?
Yes. Such datasets are called bimodal or multimodal .
4. Which measure works with categorical variables?
Mode .

5. Which measure is mathematically most useful?
Mean, because it uses all observations and is the basis for many statistical and machine learning methods.

Quick Revision	
Concept	Key Point
Statistics	Science of analyzing and interpreting data
Population	Entire group of interest
Sample	Subset used for analysis
Population Mean	μ
Sample Mean	$\bar{x}$
Population Variance	σ^2
Sample Variance	s^2
Mean	Arithmetic average
Median	Middle value
Mode	Most frequent value
Mean is affected by outliers	✔
Median is robust to outliers	✔
Mode works for categorical data	✔
Sample Variance	Uses (n-1) for unbiased estimation

What is Dispersion?
Definition
Dispersion (also called **variability** or **spread**) measures how much the data points differ from each other and from the central value.
While measures like **mean, median, and mode** tell us the **center** of the data, dispersion tells us **how scattered** the observations are.

Why is Dispersion Important?
Imagine two datasets:
Dataset A: 49, 50, 51, 50, 50

Mean = 50
Dataset B: 10, 30, 50, 70, 90

Mean = 50
Both have the **same mean**, but Dataset B is much more spread out.
A machine learning model trained on Dataset B may behave very differently because of the larger variability.

Applications in Data Science
- Detect outliers - Understand feature variability - Compare distributions - Feature scaling (Standardization) - Risk analysis - Model assumptions (many statistical models assume specific variance properties)

1. Variance
Definition
Variance measures the **average squared deviation of each observation from the mean**.
It tells us **how much the data varies around the average**.

Why Squared Difference?
If we simply summed the deviations from the mean:
-5 + 3 + 2 = 0
Positive and negative deviations cancel each other.
Squaring ensures:

- All deviations become positive.
- Larger deviations are penalized more heavily.

Population Variance Formula
$$\sigma^2 = \frac{\sum_{i=1}^N (x_i - \mu)^2}{N}$$

Where:

- x_i = observation
- μ = population mean
- N = population size

Sample Variance Formula
$$s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}$$

Why divide by (n-1)?
This is called **Bessel's Correction**.
Since the sample mean is only an estimate of the population mean, dividing by **n-1** removes the downward bias and provides an unbiased estimate of the population variance.

Properties

- Always ≥ 0 .
- Zero variance means all observations are identical.
- Larger variance means greater spread.
- Measured in **squared units**, which can make interpretation less intuitive.

Advantages

- Uses every observation.
- Foundation for many ML algorithms.
- Important in optimization and statistical inference.

Disadvantages

- Hard to interpret because of squared units.
- Sensitive to outliers.

Interview Questions
What does high variance indicate?
The observations are widely spread around the mean.

Can variance be negative?
No. Squared values cannot be negative.

Why is variance squared?
To prevent positive and negative deviations from canceling each other and to emphasize larger deviations.

2. Standard Deviation
Definition
Standard deviation is the **square root of the variance**.
It measures the average spread of the data **in the same units as the original data**, making it easier to interpret.

Formula
Population:
$$\sigma = \sqrt{\sigma^2}$$

Sample:
$$s = \sqrt{s^2}$$

Why Standard Deviation?
Variance is expressed in squared units.
Example:
If height is measured in **cm**, variance is measured in **cm²**, which is less intuitive.
Taking the square root returns the measure to the original unit (cm).

Interpretation

- Low standard deviation → data points are close to the mean.
- High standard deviation → data points are widely spread.

Applications

- Risk analysis in finance
- Standardization (Z-score normalization)
- Quality control
- Measuring consistency

Advantages

- Easy to interpret.
- Same units as the original data.
- Widely used in ML preprocessing.

Interview Questions
Which is easier to interpret: variance or standard deviation?

Standard deviation, because it uses the same units as the original data.

Can standard deviation be zero?
Yes. It is zero only if all observations are identical.

Variance vs Standard Deviation		
Feature	Variance	Standard Deviation
Unit	Squared	Original
Interpretation	Harder	Easier
Formula	Average squared deviation	Square root of variance
Use	Mathematical analysis	Practical interpretation

3. Covariance
Definition
Covariance measures **how two variables change together**.
It tells us whether two variables tend to increase or decrease together.

Formula
Population covariance:
$$Cov(X, Y) = \frac{\sum (X_i - \mu_X)(Y_i - \mu_Y)}{N}$$

Sample covariance:
$$Cov(X, Y) = \frac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{n - 1}$$

Interpretation
Positive Covariance
As one variable increases, the other tends to increase.
Example:

- Experience and Salary
- Study Hours and Marks (often)

Negative Covariance
As one increases, the other tends to decrease.
Example:

- Product Price and Demand (often)

Zero Covariance
No clear linear relationship.

Limitation
The magnitude depends on the units of the variables, making it difficult to compare across datasets.
This is why we use **Correlation**.

Interview Question
Can covariance tell the strength of a relationship?
No. It indicates only the **direction** (positive, negative, or no linear trend). The magnitude depends on the scale of the variables.

4. Correlation
Definition
Correlation measures both the **strength** and **direction** of the relationship between two variables.
Unlike covariance, it is **standardized**.

Pearson Correlation Formula
$$r = \frac{Cov(X, Y)}{\sigma_X \sigma_Y}$$

Range
$$-1 \leq r \leq 1$$

Correlation	Meaning
+1	Perfect positive linear relationship
+0.8	Strong positive relationship
+0.5	Moderate positive relationship
0	No linear relationship
-0.5	Moderate negative relationship
-0.8	Strong negative relationship
-1	Perfect negative linear relationship

Types of Correlation
Positive
Both variables move in the same direction.

Negative
One increases while the other decreases.

Zero
No linear relationship.

Correlation ≠ Causation

This is one of the **most common interview questions**. Two variables being correlated **does not mean one causes the other**.

There may be:

- A hidden variable (confounder)
- Coincidence
- Reverse causality

Always distinguish **association** from **cause-and-effect**.

Pearson vs Spearman Correlation

Pearson	Spearman
Linear relationship	Monotonic relationship
Uses actual values	Uses ranks
Sensitive to outliers	More robust to outliers
Assumes approximately normal data	No normality assumption

Applications

- Feature selection
- Detecting multicollinearity
- EDA
- Recommendation systems

Interview Questions

Difference between covariance and correlation?

Covariance measures only the direction and depends on scale.

Correlation measures both direction and strength and is always between -1 and 1.

If correlation is zero, are the variables independent?

Not necessarily. Zero correlation means **no linear relationship**. A non-linear relationship may still exist.

5. Percentiles

Definition

A percentile indicates the value below which a given percentage of observations fall.

Examples

- 90th percentile → 90% of observations are below this value.
- 50th percentile → Median.
- 95th percentile → Often used to define thresholds or detect unusually high values.

Applications

- Exam rankings
- Salary analysis
- Medical reports
- Outlier detection

6. Quartiles

Quartiles divide sorted data into **four equal parts**.

Types

Q1 (25th Percentile)

25% of observations lie below this value.

Q2 (50th Percentile)

Median.

Q3 (75th Percentile)

75% of observations lie below this value.

7. Interquartile Range (IQR)

Definition

The **Interquartile Range (IQR)** measures the spread of the **middle 50%** of the data.

Formula

$$IQR = Q3 - Q1$$

Why is IQR Useful?

Unlike variance and standard deviation, the IQR is **not heavily affected by extreme values**, making it a robust measure of spread.

Outlier Detection (1.5 × IQR Rule)

Lower Bound:

$$Q1 - 1.5 \times IQR$$

Upper Bound:

$$Q3 + 1.5 \times IQR$$

Any observation outside these bounds is typically considered an **outlier**.

Applications

- Box plots
- Robust statistics
- Data cleaning
- Outlier detection

8. Z-Score

Definition

A Z-score tells us **how many standard deviations a value is from the mean**.

Formula

$$Z = \frac{x - \mu}{\sigma}$$

Interpretation

Z-score Meaning

0	Exactly at the mean
+1	One standard deviation above the mean
-2	Two standard deviations below the mean
+3	Three standard deviations above the mean

Standard Normal Distribution

After converting values to Z-scores:

- Mean = 0
- Standard Deviation = 1

This standardized scale allows comparison across different features.

Applications

- Feature scaling
- Outlier detection (commonly $|Z| > 3$)
- Statistical testing

Interview Question

Why do we use Z-score normalization?

To bring features with different units and scales onto a comparable scale, which improves the performance of many machine learning algorithms.

9. Skewness

Definition

Skewness measures the **asymmetry** of a distribution.

Positive (Right) Skew

- Long tail on the right.
- Mean > Median > Mode.

Common examples include income or house prices.

Negative (Left) Skew

- Long tail on the left.
- Mean < Median < Mode.

Zero Skew

The distribution is symmetric.

Mean ≈ Median ≈ Mode.

Interview Question

Which measure of central tendency is preferred for highly skewed data?

The **median**, because it is less influenced by extreme values.

10. Kurtosis

Definition

Kurtosis measures the **heaviness of the tails** and the tendency of a distribution to produce extreme values. It is **not simply a measure of how "peaked" the distribution is**.

Types

Mesokurtic

- Similar tail behavior to the normal distribution.
- Excess kurtosis = 0.

Leptokurtic

- Heavier tails.
- More extreme values (outliers).

Platykurtic

- Lighter tails.
- Fewer extreme values.

Applications

- Risk analysis
- Outlier analysis
- Financial modeling

Summary Table

Concept	Measures	Range / Key Property	Sensitive to Outliers?	Common Use
Variance	Spread	≥ 0	Yes	Statistical modeling
Standard Deviation	Spread	≥ 0	Yes	Data interpretation
Covariance	Direction of joint variation	Unbounded	Yes	Relationship analysis
Correlation	Strength & direction	-1 to 1	Pearson: Yes, Spearman: Less	Feature selection
Percentile	Relative position	0–100	No	Ranking
Quartiles	Data partition	Q1, Q2, Q3	No	Distribution summary
IQR	Middle 50% spread	Q3 – Q1	No	Outlier detection
Z-score	Standardized distance from mean	Unbounded	Yes	Standardization
Skewness	Asymmetry	Negative, Zero, Positive	Yes	Distribution analysis
Kurtosis	Tail heaviness	Relative to normal	Yes	Risk & outlier analysis

🔥 Top Interview Questions

- Why do we divide sample variance by (n-1) instead of n?**
 - To obtain an unbiased estimate of the population variance (Bessel's correction).
- Difference between variance and standard deviation?**
 - Variance is the average squared deviation from the mean; standard deviation is its square root and is expressed in the original units.
- Difference between covariance and correlation?**
 - Covariance indicates the direction of joint variation and depends on scale. Correlation is standardized, ranges from -1 to 1, and measures both direction and strength.
- Can two variables have zero correlation but still be dependent?**
 - Yes. Zero correlation only rules out a **linear** relationship; a non-linear relationship may still exist.
- Why is correlation preferred over covariance?**
 - Because it is scale-independent and easier to interpret and compare.
- Why is the median preferred over the mean for skewed data?**
 - The median is robust to outliers and skewness.
- What is the purpose of the IQR?**
 - To measure the spread of the middle 50% of the data and detect outliers.
- What does a Z-score of +2 mean?**
 - The observation is two standard deviations above the mean.
- What does positive skewness indicate?**

- A longer right tail, with the mean typically greater than the median.
10. **What does high kurtosis indicate?**
- Heavier tails and a greater likelihood of extreme values (outliers).

1. What is Probability?

Definition

Probability is a numerical measure of how likely an event is to occur.

It quantifies **uncertainty**.

The value of probability always lies between
 $0 \leq P(E) \leq 1$

where:

- **0** → Impossible event
- **1** → Certain event
- Between 0 and 1 → Degree of likelihood

Interview Definition

Probability is the mathematical measure of the likelihood of an event occurring.

Why is Probability Important in Data Science?

Many ML models don't predict a class directly. Instead they predict probabilities.

Examples:

Spam Detection

Spam : 0.95

Not Spam : 0.05

Customer Churn

Will Leave : 82%

Stay : 18%

Disease Prediction

Disease : 0.91

Healthy : 0.09

Probability allows models to express **confidence** instead of just making binary decisions.

2. Basic Terminologies

Experiment

An experiment is an action whose outcome is uncertain.

Examples

- Rolling a dice
- Tossing a coin
- Predicting customer churn
- Predicting loan default

Outcome

A single possible result of an experiment.

Example

Rolling a dice

Possible outcomes

- 1
2
3
4
5
6

Each individual number is an outcome.

Sample Space (S)

Definition

The set of **all possible outcomes**.

Notation

$$S$$

Example

Rolling one dice

$$S = \{1,2,3,4,5,6\}$$

Coin Toss

$$S = \{H, T\}$$

Event (E)

An event is a subset of the sample space.

Example

Rolling an even number

$$E = \{2,4,6\}$$

Types of Events

Simple Event

Contains only one outcome.

Example

Getting exactly 4.

Compound Event

Contains multiple outcomes.

Example

Getting an even number.

Impossible Event

Never occurs.

Probability = 0

Certain Event

Always occurs.

Probability = 1

3. Probability Formula

For equally likely outcomes,

$$P(E) = \frac{\text{Number of Favorable Outcomes}}{\text{Total Outcomes}}$$

Example

Rolling an even number

Favorable outcomes

2

4

6

Total outcomes

6

Therefore

$$P(E) = \frac{3}{6} = 0.5$$

4. Probability Rules

These are among the most frequently asked interview questions.

Rule 1

Probability Range

$$0 \leq P(E) \leq 1$$

Cannot be negative.

Cannot exceed one.

Rule 2

Probability of Entire Sample Space

$$P(S) = 1$$

One of the outcomes must occur.

Rule 3

Impossible Event

$$P(\emptyset) = 0$$

Rule 4

Complement Rule

If

$$P(A)$$

Then

$$P(A^c) = 1 - P(A)$$

where A^c means "A does not occur."

Example

If

Probability of rain

0.72

Then

Probability of no rain

1-0.72=0.28

Rule 5

Addition Rule

Used for **OR** situations.

General formula

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

Why subtract?

Because the overlap is counted twice.

If A and B are mutually exclusive

$$P(A \cap B) = 0$$

then

$$P(A \cup B) = P(A) + P(B)$$

Rule 6

Multiplication Rule

Used for **AND** situations.

If events are independent,

$$P(A \cap B) = P(A)P(B)$$

Example

Coin Toss

Getting

Head AND Head

$$0.5 \times 0.5 = 0.25$$

5. Conditional Probability

One of the most important interview topics.

Definition

Conditional probability measures the probability of an event occurring **given that another event has already occurred**.

Meaning

The information available changes.

Formula

$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$

Read as

Probability of A given B

Understanding the Formula

Originally

Entire sample space

↓

After knowing B occurred

↓

Only B becomes the new sample space.

Therefore

We divide by

$$P(B)$$

Applications

Medical diagnosis

Fraud Detection

Recommendation Systems

Spam Filtering

Weather Prediction

Interview Questions

Why divide by P(B)?

Because once B has occurred, the sample space shrinks to only outcomes where B is true.

6. Independent Events

Definition

Two events are independent if one event does not affect the probability of the other.

Mathematically

$$P(A | B) = P(A)$$

or

$$P(A \cap B) = P(A)P(B)$$

Examples

Rolling a dice

and

Tossing a coin

The result of one does not affect the other.

7. Dependent Events

Definition

The occurrence of one event changes the probability of the other.

Examples

Drawing two cards **without replacement**.

The second draw depends on the first.

Independence vs Mutual Exclusivity

Very commonly asked.

Independent

Mutually Exclusive

Can happen together

Cannot happen together

$$P(A \cap B) = P(A)P(B)$$

$$P(A \cap B) = 0$$

One doesn't affect the other

If one happens, the other cannot happen

Interview Question

Can events be both independent and mutually exclusive?

No, unless one event has probability zero.

Reason:

Mutually exclusive events cannot occur together, so

$P(A \cap B) = 0$. For independent events, $P(A \cap B) =$

$P(A)P(B)$. If both probabilities are positive, these

conditions cannot both hold.

8. Bayes' Theorem ★★★★★

One of the most important topics in Data Science.

Definition

Bayes' Theorem updates the probability of an event after observing new evidence.

In simple words,

Prior belief + New evidence = Updated belief.

Formula

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}$$

Components Prior Probability Initial belief. $P(A)$	
Likelihood Probability of evidence assuming A is true. $P(B \mid A)$	
Evidence Overall probability of observing B. $P(B)$	
Posterior Probability Updated probability after observing evidence. $P(A \mid B)$	
Why Bayes is Important Used in - Naive Bayes - Spam Filtering - Medical Diagnosis - Fraud Detection - Recommendation Systems - Bayesian Statistics	
Interview Question What is Bayes' theorem used for? To update probabilities when new evidence becomes available.	
9. Random Variable Definition A random variable is a variable whose value depends on the outcome of a random experiment. Usually denoted by X	
Instead of saying Dice = 1 Dice = 2 Dice = 3 we define Random Variable X	
representing the result.	
Types of Random Variables	
Discrete Random Variable Can take only countable values. Examples - Number of customers - Number of defective products - Number of emails received	
Continuous Random Variable Can take infinitely many values within an interval. Examples - Height - Weight - Time - Temperature	
10. Expected Value (Expectation) One of the most frequently asked probability questions.	
Definition Expected value is the long-run average outcome of a random variable if the experiment is repeated many times. It is often interpreted as the "average value" you expect over repeated trials.	
Formula (Discrete) $E(X) = \sum x_i P(x_i)$	
Where: x_i = possible value $P(x_i)$ = probability of that value	
Properties Represents the center of a probability distribution.	

- It does **not** have to be one of the possible outcomes.
- It is a weighted average of all possible values.

Applications

- Risk analysis
- Decision theory
- Finance
- Reinforcement Learning (expected reward)
- Machine Learning loss functions

11. Variance of a Random Variable

Definition

Variance measures how much the values of a random variable vary around their expected value.

Formula

$$Var(X) = E[(X - E(X))^2]$$

Equivalent computational form:

$$Var(X) = E(X^2) - [E(X)]^2$$

Standard Deviation

$$\sigma = \sqrt{Var(X)}$$

Interpretation

- Small variance → values stay close to the expected value.
- Large variance → values are more spread out.

Expected Value vs Mean

This is a common interview question.

Mean	Expected Value
Computed from observed data	Computed from a probability distribution
Describes a sample or population	Describes a random variable
Based on actual observations	Based on probabilities

When you repeatedly sample from the same distribution, the sample mean approaches the expected value (Law of Large Numbers).

Common Interview Questions

1. Difference between probability and statistics?

- Probability:** Starts with a known model and predicts future outcomes.
- Statistics:** Starts with observed data and infers properties of the underlying population or process.

2. Difference between independent and mutually exclusive events?

Independent events do not influence each other. Mutually exclusive events cannot occur at the same time.

3. Why is Bayes' theorem important in Machine Learning?

It provides a principled way to update beliefs using new evidence and forms the basis of algorithms like Naive Bayes and Bayesian inference.

4. Difference between discrete and continuous random variables?

Discrete variables take countable values, while continuous variables can take any value within a range.

5. Why is expected value called an expectation?

Because it represents the average result you would expect over a very large number of repeated experiments.

6. Can expected value be a value that never actually occurs?

Yes. Since it is a weighted average, it does not need to be one of the possible outcomes.

Quick Revision Table

Topic	Key Point
Probability	Measure of likelihood (0 to 1)
Sample Space	Set of all possible outcomes
Event	Subset of the sample space
Complement Rule	$P(A^c) = 1 - P(A)$

Topic	Key Point
Addition Rule	$P(A \cup B) = P(A) + P(B) - P(A \cap B)$
Multiplication Rule	$P(A \cap B) = P(A)P(B)$ for independent events
Conditional Probability	Probability after new information is known
Independent Events	One event does not affect the other
Bayes' Theorem	Updates probabilities using evidence
Random Variable	Numerical outcome of a random experiment
Expected Value	Long-run average outcome
Variance	Spread around the expected value

1. What is a Probability Distribution?

Definition

A **Probability Distribution** describes **how the probability of a random variable is distributed across all its possible values**.

It answers questions like:

- Which values are more likely?
- Which values are rare?
- How much variation exists?
- Where is the center of the data?

Simple Intuition

Suppose we record the number of customers visiting a store every hour.

The observations might be:

12
15
13
14
15
13
16
14

A probability distribution summarizes:

- Which values occur frequently
- Which values occur rarely
- The probability of observing each value

Why is it Important?

Probability distributions help us

- Build ML models
- Estimate uncertainty
- Generate synthetic data
- Detect anomalies
- Perform hypothesis testing
- Make predictions

Types of Probability Distributions

There are two major categories.

1. Discrete Distribution

Used when the random variable takes **countable values**.

Examples

- Number of customers
- Number of defective products
- Number of emails
- Bernoulli
- Binomial
- Poisson

2. Continuous Distribution

Used when the random variable takes **any value within an interval**.

Examples

- Height
- Weight
- Temperature
- Time
- Uniform
- Normal
- Exponential

PMF vs PDF

This is a **very common interview question**.

PMF	PDF
Probability Mass Function	Probability Density Function
Used for discrete variables	Used for continuous variables
Gives probability of exact values	Gives density, not exact probability
Sum of probabilities = 1	Total area under curve = 1

Important Interview Point

For continuous variables,

$$P(X = x) = 0$$

The probability of an exact value is always zero. Instead, we calculate

$$P(a < X < b)$$

which is the **area under the curve** between two values.

1. Bernoulli Distribution ★★★★★

The simplest probability distribution.

Definition

A Bernoulli distribution models an experiment with **only two possible outcomes**.

Usually

- Success (1)
- Failure (0)

Examples

- Email is spam or not spam
- Customer buys or doesn't buy
- Loan defaults or doesn't default
- Pass or Fail

Parameter

Only one parameter

$$p = P(\text{success})$$

Failure probability

$$1 - p$$

PMF

$$P(X = x) = \begin{cases} p, & x = 1 \\ 1 - p, & x = 0 \end{cases}$$

Mean

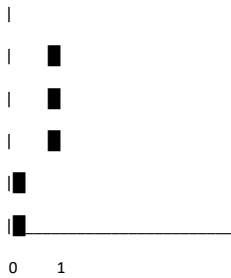
$$E(X) = p$$

Variance

$$Var(X) = p(1 - p)$$

Graph

Probability



Applications

- Binary Classification
- Logistic Regression
- Spam Detection
- Medical Diagnosis

Interview Questions

What is the difference between Bernoulli and Binomial? Bernoulli models **one trial**.

Binomial models **multiple independent Bernoulli trials**.

2. Binomial Distribution ★★★★★

One of the most asked interview topics.

Definition

The Binomial distribution models the **number of successes in n independent Bernoulli trials**.

Conditions

A Binomial experiment must satisfy:

- Fixed number of trials
- Independent trials
- Two outcomes
- Constant probability of success

If any of these assumptions fail, the Binomial model is generally not appropriate.

Parameters

$$n$$

Number of trials

$$p$$

Probability of success

PMF

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

Where

$$\binom{n}{k}$$

is the number of ways to choose k successes from n trials.

Mean

$$np$$

Variance

$$np(1 - p)$$

Shape

- Symmetric when $p = 0.5$
- Right-skewed when $p < 0.5$
- Left-skewed when $p > 0.5$

Applications

- Number of customers who click an ad
- Number of defective products
- Number of successful transactions
- Number of accepted applications

Interview Questions

Why is independence important?

Because the probability of success should remain constant across trials. If one trial affects another, the Binomial assumptions are violated.

Bernoulli vs Binomial

Bernoulli	Binomial
One trial	Multiple trials
Two outcomes	Counts total successes
Parameter p	Parameters n and p

3. Poisson Distribution ★★★★★

Very common in interviews.

Definition

Poisson models the **number of times an event occurs within a fixed interval of time, area, distance, or volume**.

Examples:

- Emails received per hour
- Customers arriving per minute
- Website requests per second
- Machine failures per month

Parameter

$$\lambda$$

Average number of events in the interval.

PMF

$$P(X = k) = \frac{e^{-\lambda} \lambda^k}{k!}$$

Mean

$$\lambda$$

Variance

$$\lambda$$

This is an important interview property:

Mean = Variance

Assumptions

- Events occur independently.
- Average rate remains constant.
- Two events cannot occur at exactly the same instant (in the idealized model).

Applications

- Fraud detection
- Network traffic
- Insurance claims
- Manufacturing defects
- Call center arrivals

Interview Questions

Difference between Binomial and Poisson?

Binomial	Poisson
Fixed number of trials	No fixed number of trials
Counts successes	Counts event occurrences
Parameters n and p	Parameter λ
Mean = np	Mean = λ

4. Uniform Distribution

Definition

Every value within a specified interval is **equally likely**.

Continuous Uniform Distribution

Suppose

$$a \leq X \leq b$$

Then every value between a and b has the same probability density.

PDF

$$f(x) = \frac{1}{b - a}$$

Mean

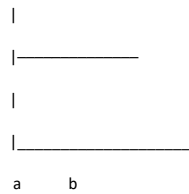
$$\frac{a + b}{2}$$

Variance

$$\frac{(b - a)^2}{12}$$

Graph

Density



Applications

- Random number generation
- Simulation
- Monte Carlo methods
- Random initialization

5. Normal (Gaussian) Distribution ★★★★★

The most important probability distribution in statistics and ML.

Definition

A Normal Distribution is a continuous probability distribution that is

- Symmetric
- Bell-shaped
- Completely described by its mean and standard deviation

Why is it Called Gaussian?

Named after the mathematician **Carl Friedrich Gauss**, who made major contributions to its study.

Normal Distribution and Gaussian Distribution refer to the same distribution.

PDF

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

You are rarely expected to derive this formula in interviews, but you should know what each parameter represents.

Parameters

Mean μ

Controls the center.
Standard deviation

σ

Controls the spread.

Properties

Symmetric

Mean = Median = Mode

Bell Shaped

Most observations cluster near the center.

Total Area

Total area under the curve = 1

Tails Never Touch the X-axis

The curve approaches the axis asymptotically.

68–95–99.7 Rule ★★★★★★

Also called the **Empirical Rule**.

Range Data Covered

$\mu \pm 1\sigma$ 68%

$\mu \pm 2\sigma$ 95%

$\mu \pm 3\sigma$ 99.7%

Why is the Normal Distribution Important?

Many natural and measurement-based phenomena approximately follow it because of the **Central Limit Theorem (CLT)**, which states that the sampling distribution of the sample mean tends toward a normal distribution as sample size increases (under fairly general conditions).

Examples:

- Heights
- Measurement errors
- IQ scores (approximately)
- Many biological measurements

Applications

- Statistical inference
- Hypothesis testing
- Confidence intervals
- Standardization (Z-score)
- Linear Regression assumptions (errors/residuals)

Interview Questions

Why is Normal Distribution important?

Because many statistical methods either assume normality or rely on the Central Limit Theorem, making it central to inference and modeling.

Why is it called a bell curve?

Because of its symmetric, bell-like shape.

Standard Normal Distribution

A special case of the Normal Distribution.

Mean 0

Standard deviation 1

Every normal variable can be converted to a standard normal variable using the Z-score.

6. Exponential Distribution

Definition

The Exponential Distribution models the **time between consecutive events** in a Poisson process.

If Poisson answers:

"How many events occur?"

Exponential answers:

"How long until the next event?"

Parameter

λ

Rate of occurrence.

PDF

$$f(x) = \lambda e^{-\lambda x}$$

for $x \geq 0$.

Mean

$$\frac{1}{\lambda}$$

Variance

$$\frac{1}{\lambda^2}$$

Memoryless Property ★★★★★★

This is the defining property of the Exponential Distribution.

The probability of waiting an additional amount of time does **not** depend on how long you have already waited. Examples include idealized models of:

- Time until the next customer arrives
- Time until the next network packet
- Time between machine failures (under constant failure rate assumptions)

Applications

- Queueing systems
- Reliability engineering
- Survival analysis
- Network modeling

Relationship Between Distributions

This is another favorite interview topic.

Bernoulli



Binomial



Poisson (approximation to Binomial when n is very large and p is very small, with $\lambda = np$)



Exponential

(Time between Poisson events)

Normal Distribution is not derived from these, but through the **Central Limit Theorem**, many sums and averages of random variables become approximately normal.

Comparison Table

Distribution	Variable Type	Parameters	Mean	Variance	Typical Use
Bernoulli	Discrete	p	p	$p(1-p)$	One success/failure trial
Binomial	Discrete	n, p	np	$np(1-p)$	Number of successes
Poisson	Discrete	λ	λ	λ	Event counts in a fixed interval
Uniform	Continuous	a, b	$\frac{(a+b)}{2}$	$\frac{(b-a)^2}{12}$	Equal likelihood over an interval
Normal (Gaussian)	Continuous	μ, σ	μ	σ^2	Natural measurements, inference
Exponential	Continuous	λ	$1/\lambda$	$1/\lambda^2$	Waiting time between Poisson events

Which Distribution Should You Use?

Situation	Distribution
Binary outcome (yes/no)	Bernoulli
Number of successes in fixed trials	Binomial
Number of events in fixed time/space	Poisson
Every value equally likely in an interval	Uniform
Symmetric continuous data	Normal
Time until the next event	Exponential

Top Interview Questions

1. Difference between Bernoulli and Binomial?

- Bernoulli models a single binary trial.

- Binomial models the number of successes across multiple independent Bernoulli trials.

2. When can Poisson approximate Binomial?

When:

- n is large,
- p is small,
- and $\lambda = np$ remains moderate.

3. Why is the Normal Distribution so important?

Because many statistical methods rely on it directly or on the Central Limit Theorem, which makes sample means approximately normal.

4. What is the memoryless property?

The future waiting time is independent of how much time has already elapsed. The Exponential Distribution is the most common continuous distribution with this property.

5. Why does Poisson have equal mean and variance?

It is a defining mathematical property of the Poisson distribution. In practice, if observed variance is much larger than the mean (overdispersion), a simple Poisson model may not fit well.

6. What is the difference between PMF and PDF?

- PMF gives probabilities for exact values of discrete variables.
- PDF gives probability density for continuous variables; probabilities are obtained by calculating the **area under the curve** over an interval.

1. What is Statistical Inference?

Definition

Statistical Inference is the process of using **sample data** to make conclusions about an entire population.

Since collecting data from an entire population is often impossible or expensive, we collect a **sample** and use statistical techniques to infer population characteristics.

Why is Statistical Inference Needed?

Suppose a company has **20 million users**.

Instead of surveying every user, it surveys **10,000 users** and uses the results to estimate the behavior of the entire user base.

This process is called **statistical inference**.

Two Main Types of Statistical Inference

1. Estimation

Estimate unknown population parameters.

Examples:

- Average salary
- Average customer spending
- Average delivery time

2. Hypothesis Testing

Test whether a claim about the population is supported by the sample data.

Example:

"Did the new recommendation algorithm increase user engagement?"

2. What is Hypothesis Testing?

Definition

Hypothesis testing is a statistical method used to determine whether there is **enough evidence in a sample** to support or reject a claim about a population. Notice the wording:

We **do not prove** hypotheses.

We only determine whether there is sufficient evidence **against** the null hypothesis.

Interview Definition

Hypothesis testing is a statistical procedure that uses sample data to decide whether there is sufficient evidence to reject a population claim.

Why Do We Need Hypothesis Testing?

Imagine a company launches a new checkout page.

Conversion rate changes from

Old = 6.1%

New = 6.4%

Question:

Did the new page actually improve conversions?

Or

Did the increase happen just due to random chance?

Hypothesis testing answers this question.

3. Basic Terminology

Null Hypothesis (H_0)

Definition

The Null Hypothesis represents the default assumption . It usually states No effect or No difference or No relationship
Examples Medicine H_0 : New medicine is not better than the old one.
Machine Learning H_0 : The new model performs the same as the old model.
Business H_0 : Changing the website does not increase sales.
Think of H_0 as the "status quo." It is assumed true until strong evidence suggests otherwise.
Alternative Hypothesis (H_1 or H_a) Definition The Alternative Hypothesis is the claim we want to investigate. It states that there is an effect, difference, or relationship .
Examples Medicine H_1 : New medicine performs better.
Business H_1 : New website increases sales.
Machine Learning H_1 : New recommendation model improves CTR.
Interview Question Why do we assume the null hypothesis is true? Because statistical testing is designed to determine whether there is enough evidence to reject the default assumption.
4. Steps of Hypothesis Testing This workflow is frequently asked in interviews. Problem ↓ State H_0 and H_1 ↓ Choose significance level (α) ↓ Collect sample ↓ Calculate test statistic ↓ Calculate p-value ↓ Decision ↓ Business conclusion
5. Significance Level (α) Definition The significance level is the maximum probability of making a Type I Error that we are willing to accept. Usually $\alpha = 0.05$ Sometimes 0.01 or 0.10 depending on the application.
Interpretation $\alpha = 0.05$ means:

We accept a 5% risk of incorrectly rejecting a true null hypothesis . It is not the probability that the null hypothesis is true.
6. p-value ★★★★★ One of the most misunderstood interview topics.
Definition The p-value is the probability of observing data at least as extreme as the observed sample , assuming that the null hypothesis is true . This definition is the one interviewers expect.
Important Interpretation A small p-value means: The observed data would be unlikely if the null hypothesis were true. This provides evidence against the null hypothesis.
Decision Rule If $p < \alpha$ Reject H_0 . If $p \geq \alpha$ Fail to reject H_0 . Notice: We say fail to reject , not accept the null hypothesis.
Common Misconceptions (Very Important) ✗ Wrong p-value is the probability that H_0 is true. ✗ Wrong p-value tells how important the result is. ✗ Wrong A smaller p-value always means a larger practical effect. ✓ Correct It measures how compatible the observed data is with the null hypothesis.
Interview Question If $p = 0.03$ and $\alpha = 0.05$ Answer Since $0.03 < 0.05$ Reject H_0 . There is statistically significant evidence against the null hypothesis at the 5% significance level.
7. Confidence Interval Definition A confidence interval is a range of plausible values for an unknown population parameter. Instead of giving one estimate, we provide an interval. Example Estimated average salary $\$58,000 \pm \$2,000$ 95% Confidence Interval $\$56,000 - \$60,000$
Correct Interpretation A 95% confidence interval means: If we repeated the sampling process many times and constructed a confidence interval each time using the same method, approximately 95% of those intervals would contain the true population parameter . It does not mean there is a 95% probability that the true parameter lies inside the specific interval you calculated.
Interview Question If the confidence interval for the difference between two means includes 0 , then there is no statistically significant difference at the corresponding confidence level.
8. Errors in Hypothesis Testing Every decision can be correct or incorrect.
Type I Error (False Positive) Rejecting H_0 when H_0 is actually true. Example Concluding a drug works when it actually doesn't. Probability α
Type II Error (False Negative)

Failing to reject H_0 when H_0 is actually false.

Example
Concluding a drug doesn't work when it actually does.

Probability
$$\beta$$

Power of Test

Definition
Power is the probability of correctly rejecting a false null hypothesis.

Formula
$$Power = 1 - \beta$$

Higher power is better because it means you're more likely to detect a real effect.
Power generally increases with:

- Larger sample size
- Larger true effect size
- Lower variability
- Higher significance level (though this increases Type I error risk)

Type I vs Type II Error

Type I	Type II
False Positive	False Negative
Reject true H_0	Fail to reject false H_0
Probability = α	Probability = β

9. One-Tailed vs Two-Tailed Test

One-Tailed Test

Used when the direction matters.
Examples
New model is **better**
New model is **faster**
Sales **increase**

Example hypothesis
$$H_1: \mu > 50$$

or
$$H_1: \mu < 50$$

Two-Tailed Test

Used when **any difference** matters.
Example
Has the average changed?
Not
Has it increased?

Hypothesis
$$H_1: \mu \neq 50$$

Interview Tip

Use a **one-tailed test** only when the research question was defined in one direction **before** seeing the data.

10. z-test

Purpose

Compares sample statistics when the population variance is known or the sample size is sufficiently large (commonly $n \geq 30$).

When to Use

- Data approximately normal, or large sample (CLT)
- Population standard deviation known (strictly)
- Large sample

Formula

$$Z = \frac{\bar{x} - \mu}{\sigma / \sqrt{n}}$$

Applications

- Average salary
- Average delivery time
- Mean customer rating

11. t-test ★★★★★

One of the most important interview topics.

Purpose

Compare means when

- Population variance is unknown

Usually used for **small samples**, though it is also appropriate for larger samples when the population variance is unknown.

Formula

$$t = \frac{\bar{x} - \mu}{s/\sqrt{n}}$$

Why t instead of z?

Because

Population standard deviation is unknown.

The t-distribution accounts for the extra uncertainty introduced by estimating the standard deviation from the sample.

Types of t-test

One Sample t-test

Compare sample mean with known population mean.

Independent t-test

Compare two independent groups.

Example
Old website
vs
New website

Paired t-test

Same individuals before and after treatment.
Examples

- Before vs after training
- Before vs after marketing campaign
- Before vs after medication

z-test vs t-test

z-test	t-test
Population standard deviation known (or large-sample approximation)	Population standard deviation unknown
Uses Normal distribution	Uses t-distribution
Best for large samples	Especially useful for smaller samples
Narrower tails	Heavier tails

12. Chi-Square Test ★★★★★

One of the most commonly asked interview topics.

Purpose

Used with **categorical data**.

Applications

- Test independence
- Test association
- Goodness of fit

Example Questions

Is Gender related to Product Choice?

Is Device Type related to Conversion?

Is Region related to Churn?

Chi-Square Statistic

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

Where:

- O = Observed frequency
- E = Expected frequency under H_0

Assumptions

- Observations are independent.
- Data are counts (frequencies).
- Expected frequencies should generally be sufficiently large (a common guideline is at least 5 in most cells).

13. ANOVA (Analysis of Variance) ★★★★★

Purpose

Compare the means of **three or more groups**.

Interview Question

Why not perform multiple t-tests?

Because doing many t-tests increases the probability of making a Type I Error.

ANOVA controls this by testing all groups simultaneously.

Null Hypothesis

All group means are equal.

Alternative Hypothesis

At least one group mean is different.

Test Statistic

ANOVA uses the **F-statistic**, which compares:

- Variability **between** groups
- Variability **within** groups

A larger F-statistic suggests stronger evidence that at least one group mean differs.

Applications

Compare:

- Three ML models
- Multiple marketing campaigns
- Several pricing strategies

Important Note

ANOVA tells you **whether** at least one group differs, **not which one**.

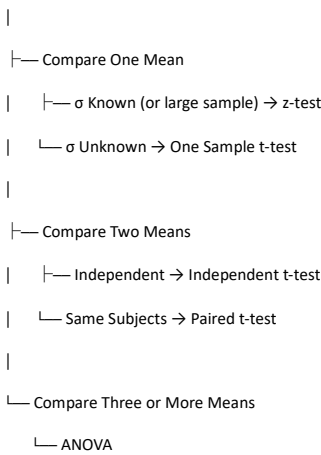
To identify the differing groups, use **post-hoc tests** (e.g., Tukey's HSD).

Choosing the Correct Test ★★★★★

Problem	Test
One mean vs population mean (σ known or large sample)	z-test
One mean vs population mean (σ unknown)	One-sample t-test
Two independent means	Independent t-test
Before vs after on same subjects	Paired t-test
Relationship between categorical variables	Chi-Square Test of Independence
Compare means of 3 or more groups	ANOVA

Complete Interview Decision Flow

Numerical Target



Categorical Variables

↓

Chi-Square Test

Top Interview Questions

1. What is the difference between statistical significance and practical significance?

- Statistical significance** means the observed effect is unlikely to be due to random chance (based on the chosen significance level).
- Practical significance** asks whether the effect is **large enough to matter in the real world**.

A tiny improvement can be statistically significant with a very large sample but still have little business value.

2. What does a p-value of 0.01 mean?

It means that **if the null hypothesis were true**, the probability of observing data at least as extreme as what was observed is **1%**.

It does **not** mean there is a 1% chance that the null hypothesis is true.

3. Why don't we say "accept H_0 "?

Because failing to find sufficient evidence against H_0 is not the same as proving H_0 is true.

4. Why use ANOVA instead of multiple t-tests?

Multiple t-tests inflate the probability of making a Type I Error. ANOVA controls this by evaluating all group means simultaneously.

5. What is the difference between confidence level and confidence interval?

- Confidence level** (e.g., 95%) describes the long-run performance of the interval estimation procedure.
- Confidence interval** is the specific range calculated from your sample.

6. What is statistical power?

Power is the probability of correctly detecting a true effect, i.e., correctly rejecting a false null hypothesis.

Quick Revision

Concept	Key Point
Statistical Inference	Draw conclusions about a population from a sample
Hypothesis Testing	Decide whether sample evidence contradicts H_0
H_0	Default assumption (no effect/difference)
H_1	Research claim (effect/difference exists)
α	Acceptable Type I Error rate
p-value	Probability of data this extreme assuming H_0 is true
Confidence Interval	Range of plausible values for a population parameter
Type I Error	False Positive
Type II Error	False Negative
Power	Probability of detecting a real effect
z-test	Mean comparison (σ known or large-sample approximation)
t-test	Mean comparison (σ unknown)
Chi-Square	Categorical data
ANOVA	Compare 3+ group means
Statistical ≠ Practical Significance	Always interpret both

